

Санкт-Петербургский политехнический университет Петра
Великого
Институт компьютерных наук и кибербезопасности
Высшая школа программной инженерии

П Р А К Т И Ч Е С К А Я Р А Б О Т А № 4

Исследование и демонстрация тестирования идиомпотентности
и повторяемости запросов в сетевых API на Java (Spring Boot)

по дисциплине «Методы тестирования программного обеспечения»

Выполнил
студент гр. 5130903/30303

В. А. Плахотников

Руководитель
старший преподаватель
ВШ ПИ

В. А. Пархоменко

Содержание

1	Введение	3
1.1	Актуальность	3
1.2	Постановка проблемы	3
1.3	Задачи	3
2	Методы	4
2.1	Математическая постановка задачи идемпотентности	4
2.2	Классификация HTTP-методов по идемпотентности	6
2.3	Паттерн Idempotency-Key	7
2.4	Optimistic Locking как механизм идемпотентности	7
2.5	Псевдокод алгоритма проверки идемпотентности	8
2.6	Пошаговое выполнение алгоритма на иллюстративных данных	9
2.7	Типы организации контроллеров в Spring и Java-экосистеме	10
2.7.1	@RestController (аннотационный подход)	11
2.7.2	Functional Endpoints (WebMvc.fn)	11
2.7.3	Spring Data REST	11
2.7.4	Spring-WS (SOAP, контракт-первый)	12
2.7.5	gRPC (HTTP/2 + protobuf)	12
2.7.6	GraphQL (single endpoint, schema-first)	13
2.8	Транспорт-агностичный снэпшот ответа	14
2.9	Средства тестирования	14
3	Результаты	15
3.1	Описание разработанного программного обеспечения	15
3.1.1	Структура проекта	16
3.1.2	Схема базы данных	16
3.1.3	Компоненты механизма идемпотентности	17
3.1.4	Профили конфигурации	17
3.2	Результаты тестирования (профиль idempotent)	18
3.3	Тест-кейсы по контроллерам	18
3.3.1	@RestController (OrderIdempotencyTest)	18
3.3.2	Functional Endpoints (PaymentIdempotencyTest)	19
3.3.3	Spring Data REST (ProductIdempotencyTest)	19
3.3.4	Импорт данных (DataImportTest)	19
3.3.5	SOAP (SoapIdempotencyTest)	20
3.3.6	gRPC (GrpcIdempotencyTest)	20
3.3.7	GraphQL (GraphQLIdempotencyTest)	20
3.4	Результаты профиля non-idempotent	21

4	Обсуждение	22
4.1	Сравнение подходов к обеспечению идемпотентности	22
4.2	Анализ результатов по транспортам	22
4.3	Идемпотентность как конвенция: почему нельзя «всё делать через GET»	24
4.4	Оптимистичная блокировка и границы её применимости для идемпотентности	25
4.5	Значимость Testcontainers в тестировании идемпотентности .	26
4.6	Рекомендации для production	27
5	Заключение	28
6	Инструкция по установке и запуску	29
6.1	Требования	29
6.2	Запуск приложения	29
6.3	Запуск тестов	29
6.4	Пример использования API (все четыре транспорта)	30
	Приложение А. Листинги исходного кода	33

1 Введение

1.1 Актуальность

В современных распределённых системах REST API является основным способом взаимодействия между сервисами. При обработке HTTP-запросов неизбежно возникают ситуации, когда запрос может быть отправлен повторно: из-за сетевых таймаутов, retry-политик, дублирования на уровне балансировщиков нагрузки или ошибок клиента. Без механизмов идемпотентности повторная обработка запроса приводит к критическим последствиям: дублированию платежей, двойному списанию средств, созданию дубликатов записей в базе данных [1].

Тестирование идемпотентности особенно важно в финансовых и e-commerce системах, где повторная обработка транзакции приводит к прямым финансовым потерям. Компании, такие как Stripe, реализуют паттерн Idempotency-Key как обязательный элемент API [2].

1.2 Постановка проблемы

Проблема заключается в необходимости обеспечения корректного поведения сервиса при повторных запросах. HTTP-спецификация [3] определяет, какие методы должны быть идемпотентными (GET, PUT, DELETE), а какие — нет (POST). Однако на практике реализация идемпотентности требует дополнительных механизмов на уровне приложения, причём конкретный механизм существенно зависит от выбранного транспорта (REST, SOAP, gRPC, GraphQL) и от стиля организации контроллера в Spring.

Цель данной работы — исследовать и продемонстрировать методы тестирования идемпотентности и повторяемости запросов на примере Spring Boot приложения с шестью кардинально различными типами организации контроллеров, охватывающими четыре транспортных протокола: HTTP/REST (в трёх стилях), SOAP, gRPC и GraphQL.

1.3 Задачи

1. Формализовать понятие идемпотентности HTTP-запросов математически.
2. Разработать сервис с шестью различными типами контроллеров, объединёнными единым механизмом идемпотентности:
 - аннотационный `@RestController` (по образцу Spring Guides [4]);
 - функциональные эндпоинты `RouterFunction` (по образцу Spring Framework [5]);

- автогенерируемые эндпоинты Spring Data REST [6];
 - SOAP-эндпоинт на Spring-WS [7] (контракт-первый, XSD $\rightarrow$ WSDL);
 - gRPC-сервис на базе `grpc-spring-boot-starter` [8] и сгенерированных protobuf-стабов;
 - GraphQL-контроллер на `spring-boot-starter-graphql` [9] со схемой SDL.
3. Реализовать механизмы обеспечения идемпотентности (Idempotency-Key, optimistic locking, unique constraints) и продемонстрировать их переиспользование между транспортами: единый `IdempotencyService` обслуживает HTTP-фильтр (REST/SOAP/GraphQL) и gRPC `ServerInterceptor`.
 4. Сделать сам механизм *транспорт-агностичным*: модернизировать существующий фильтр так, чтобы он корректно реплеил ответы любого формата (JSON, XML/SOAP, бинарный protobuf).
 5. Разработать набор интеграционных тестов с использованием Testcontainers [10].
 6. Продемонстрировать поведение системы без механизмов идемпотентности через отдельный Spring-профиль, причём для каждого из шести транспортов.

2 Методы

2.1 Математическая постановка задачи идемпотентности

Алгебраическое определение [11, § I.1]. Оператор $f: X \rightarrow X$ называется *идемпотентным*, если выполняется

$$f \circ f = f, \quad \text{т. е. } f(f(x)) = f(x) \quad \forall x \in X. \quad (1)$$

Идемпотентность HTTP-метода [3, § 9.2.2]. HTTP-метод считается *идемпотентным*, если результирующий эффект на сервере от многократного выполнения идентичных запросов с этим методом совпадает с эффектом от однократного выполнения такого запроса¹. Эквивалентную формулировку даёт П. Хелланд [1]: «multiple invocations are identical to exactly one invocation».

¹Дословно (RFC 9110, § 9.2.2): «A request method is considered “idempotent” if the intended effect on the server of multiple identical requests with that method is the same as the effect for a single such request».

Формализация. Два приведённых выше определения (алгебраическое и текстовое НТТР) объединяются, если рассматривать обработку запроса как оператор над состоянием сервера. Введём обозначения:

- S — множество состояний системы (совокупность данных в БД);
- R — множество НТТР-запросов;
- $O: S \times R \rightarrow S \times \text{Response}$ — операция обработки запроса;
- $\pi_1(O(s, r))$ — проекция на новое состояние.

Применяя алгебраическое определение к оператору $s \mapsto \pi_1(O(s, r))$ при фиксированном r , получаем формальный критерий: НТТР-метод m идемпотентен тогда и только тогда, когда для любого состояния $s \in S$ и запроса r с методом m :

$$\pi_1(O^n(s, r)) = \pi_1(O(s, r)), \quad \forall n \geq 1, \quad (2)$$

где O^n обозначает n -кратное применение операции, при этом каждое следующее применение использует состояние, полученное на предыдущем шаге:

$$O^1(s, r) = O(s, r), \quad O^{n+1}(s, r) = O(\pi_1(O^n(s, r)), r). \quad (3)$$

Эта формула — авторская формализация в рамках данной работы; в первоисточниках [1; 3] идемпотентность НТТР формулируется словесно, поскольку их жанр (стандарт, обзорная статья) формальных конструкций над состоянием не предполагает.

Свойства. Согласно RFC 9110 [3]:

- **Идемпотентные методы:** GET, HEAD, PUT, DELETE, OPTIONS
- **Неидемпотентные методы:** POST, PATCH
- **Safe (безопасные) методы:** GET, HEAD, OPTIONS — не изменяют состояние

Формально, для safe-метода m :

$$\pi_1(O(s, r)) = s, \quad \forall s \in S, r \text{ с методом } m \quad (4)$$

Из этого следует, что каждый safe-метод является идемпотентным, но не наоборот: DELETE изменяет состояние при первом вызове (удаляет ресурс), однако повторные вызовы не изменяют его далее — ресурс уже отсутствует.

Задача. Для метода POST, который не является идемпотентным по определению, необходимо обеспечить идемпотентность через дополнительные механизмы. Обозначим k — ключ идемпотентности. Тогда модифицированная операция O' :

$$O'(s, r, k) = \begin{cases} O(s, r), & \text{если } k \notin \text{KeyStore}(s) \\ (s, \text{CachedResponse}(k)), & \text{если } k \in \text{KeyStore}(s) \end{cases} \quad (5)$$

что гарантирует: $\pi_1(O'^n(s, r, k)) = \pi_1(O'(s, r, k))$, $\forall n \geq 1$.

Доказательство. Пусть $s_1 = \pi_1(O'(s, r, k))$. При первом вызове $k \notin \text{KeyStore}(s)$, поэтому выполняется $O(s, r)$ и ключ k регистрируется в хранилище. При втором вызове $k \in \text{KeyStore}(s_1)$, поэтому состояние не изменяется: $\pi_1(O'(s_1, r, k)) = s_1$. По индукции, для любого $n \geq 1$: $\pi_1(O'^n(s, r, k)) = s_1 = \pi_1(O'(s, r, k))$. $\square$

2.2 Классификация HTTP-методов по идемпотентности

Спецификация HTTP (RFC 9110 [3]) определяет свойства методов следующим образом:

Таблица 1: Свойства HTTP-методов (RFC 9110)

Метод	Safe	Идемп.	Тело	Семантика
GET	Да	Да	Нет	Получение представления ресурса
HEAD	Да	Да	Нет	Получение заголовков без тела
PUT	Нет	Да	Да	Полная замена состояния ресурса
DELETE	Нет	Да	Нет	Удаление ресурса
POST	Нет	Нет	Да	Создание или произвольная обработка
PATCH	Нет	Нет	Да	Частичное обновление ресурса

Различие между PUT и PATCH имеет принципиальное значение для идемпотентности. PUT заменяет ресурс целиком, поэтому повторный вызов с теми же данными не меняет состояние. PATCH применяет дельту: например, операция «увеличить количество на 1» при повторном применении даст другой результат, поэтому PATCH не является идемпотентным в общем случае.

2.3 Паттерн Idempotency-Key

Согласно IETF-стандарту [12], клиент передаёт уникальный ключ в заголовке `Idempotency-Key`. Сервер сохраняет этот ключ вместе с кэшированным ответом. При повторном запросе с тем же ключом сервер возвращает кэшированный ответ без повторного выполнения операции.

Данный паттерн широко применяется в production-системах. В частности, платёжная система Stripe требует заголовков `Idempotency-Key` для всех POST-запросов, связанных с финансовыми операциями [2]. Это предотвращает дублирование платежей при retry.

Жизненный цикл ключа идемпотентности:

1. **Генерация** — клиент генерирует UUID перед первым запросом.
2. **Регистрация** — сервер сохраняет ключ в хранилище (БД) до обработки запроса.
3. **Обработка** — выполняется бизнес-логика, результат кэшируется.
4. **Повторное использование** — при повторном запросе возвращается кэш.
5. **Истечение** — по прошествии TTL (обычно 24 часа) ключ удаляется.

2.4 Optimistic Locking как механизм идемпотентности

Для метода PUT идемпотентность может быть усилена через *optimistic locking* [13]. Каждая сущность содержит поле версии v . При обновлении клиент передаёт текущую версию, и сервер проверяет совпадение:

$$\text{Update}(s, r, v) = \begin{cases} (s', 200 \text{ OK}), & \text{если } v = \text{CurrentVersion}(s) \\ (s, 409 \text{ Conflict}), & \text{если } v \neq \text{CurrentVersion}(s) \end{cases} \quad (6)$$

В Spring Data REST это реализуется через аннотацию `@Version`: Hibernate автоматически добавляет проверку версии в SQL-запрос UPDATE, а Spring генерирует HTTP-заголовок ETag из значения версии. Клиент передаёт ETag через заголовок `If-Match`, и при несовпадении версий сервер возвращает `412 Precondition Failed`.

Это предотвращает проблему *lost update* — ситуацию, когда два клиента одновременно читают одну версию ресурса, модифицируют его и сохраняют, при этом изменения первого клиента теряются.

2.5 Псевдокод алгоритма проверки идиempотентности

Алгоритм 1 Проверка идиempотентности HTTP-запроса

Вход: HTTP-запрос *request*, хранилище ключей *KeyStore*

Выход: HTTP-ответ *response*

```
1: Функция CHECKIDEMPOTENCY(request)
2:   если request.method  $\neq$  POST тогда
3:     вернуть PROCESSREQUEST(request)      ▷ GET/PUT/DELETE
        идиempотентны
4:   конец если
5:   key  $\leftarrow$  request.headers["Idempotency-Key"]
6:   если key =  $\emptyset$  тогда
7:     вернуть 400 Bad Request                ▷ Ключ обязателен для POST
8:   конец если
9:   cached  $\leftarrow$  KeyStore.find(key)
10:  если cached  $\neq \emptyset$  и cached.response  $\neq \emptyset$  тогда
11:    вернуть cached.response                ▷ Повторный запрос —
        кэшированный ответ
12:  конец если
13:  если cached  $\neq \emptyset$  и cached.response =  $\emptyset$  тогда
14:    вернуть 409 Conflict                    ▷ Запрос уже обрабатывается
15:  конец если
16:  KeyStore.register(key, request.method, request.path)
17:  response  $\leftarrow$  PROCESSREQUEST(request)
18:  KeyStore.saveResponse(key, response)
19:  вернуть response
20: конец Функция
```

Сложность алгоритма. Поиск ключа в хранилище выполняется за $O(1)$ при использовании индекса (B-tree) на поле `idempotency_key`. Регистрация и сохранение ответа — по одной операции INSERT/UPDATE. Общая сложность: $O(1)$ дополнительных операций к каждому POST-запросу.

Обработка конкурентных запросов. Строка 15 алгоритма предотвращает гонку (race condition): если два запроса с одним ключом поступают одновременно, только первый успешно регистрирует ключ (благодаря UNIQUE constraint в БД), второй получит ошибку `DataIntegrityViolationException` и вернёт 409 Conflict.

2.6 Пошаговое выполнение алгоритма на иллюстративных данных

Сценарий: клиент создаёт заказ (POST /api/orders) и из-за таймаута повторяет запрос.

Входные данные:

- Запрос: POST /api/orders, тело: {description: "Ноутбук amount: 89999.99}
- Заголовок: Idempotency-Key: abc-123
- Начальное состояние: таблица orders пуста, KeyStore пуст

Первый запрос (шаг за шагом):

1. Метод = POST $\Rightarrow$ проверяем Idempotency-Key (строка 2 алгоритма)
2. $key = \text{"abc-123"} \neq \emptyset \Rightarrow$ продолжаем (строка 6)
3. $KeyStore.find(\text{"abc-123"}) = \emptyset \Rightarrow$ ключ новый (строка 9)
4. $KeyStore.register(\text{"abc-123"}, \text{POST}, \text{"api/orders"})$ (строка 17)
5. Выполняем **ProcessRequest**: INSERT INTO orders $\Rightarrow$ заказ создан (id=e7f1...) (строка 18)
6. $KeyStore.saveResponse(\text{"abc-123"}, 201, \text{"{id: e7f1...}"})$ (строка 19)
7. Ответ: 201 Created, тело: {id: "e7f1... description: "Ноутбук"}

Повторный запрос (retry после таймаута):

1. Метод = POST $\Rightarrow$ проверяем Idempotency-Key (строка 2)
2. $key = \text{"abc-123"} \neq \emptyset \Rightarrow$ продолжаем (строка 6)
3. $KeyStore.find(\text{"abc-123"}) \neq \emptyset, cached.response \neq \emptyset$ (строка 10)
4. Возвращаем кэшированный ответ: 201 Created, тело: {id: "e7f1..."} (строка 11)
5. **Заказ НЕ дублируется** — в таблице по-прежнему одна запись

Трассировка состояний:

Таблица 2: Пошаговая трассировка алгоритма на иллюстративных данных

Шаг	Действие	KeyStore	orders
0	Начальное состояние	$\emptyset$	0 записей
1	Получен key="abc-123"	$\emptyset$	0 записей
2	find("abc-123") = $\emptyset$	$\emptyset$	0 записей
3	register("abc-123")	{abc-123: pending}	0 записей
4	INSERT INTO orders	{abc-123: pending}	1 запись
5	saveResponse("abc-123", 201)	{abc-123: 201, body}	1 запись
6	Retry: key="abc-123"	{abc-123: 201, body}	1 запись
7	find("abc-123") $\neq \emptyset$	{abc-123: 201, body}	1 запись
8	return cached(201)	{abc-123: 201, body}	1 запись

2.7 Типы организации контроллеров в Spring и Java-экосистеме

В данной работе используются **шесть кардинально различных подходов** к организации серверных контроллеров, охватывающих четыре транспортных протокола (таблица 3). Первые три — разные стили HTTP/REST в Spring MVC; следующие три — принципиально иные протоколы (SOAP, gRPC, GraphQL).

Таблица 3: Сравнение шести типов контроллеров

Тип	Транспорт	Маршрутизация / обработчик	Механизм идиempотентности
@RestController	HTTP/REST (JSON)	Аннотации @GetMapping, метод контроллера	Idempotency-Key (HTTP-фильтр)
Functional Endpoints	HTTP/REST (JSON)	RouterFunction HandlerFunction	→ Idempotency-Key (тот же фильтр)
Spring Data REST	HTTP/HAL+JSON	Автогенерация @RepositoryRestResource	из @Version + ETag
Spring-WS @Endpoint	SOAP / HTTP+XML	@PayloadRoot, MessageDispatcherServlet	Idempotency-Key (тот же фильтр)
GraphQL @Controller	GraphQL / HTTP	@QueryMapping @MutationMapping, endpoint	/ Idempotency-Key (тот же фильтр) single
@GrpcService	gRPC / HTTP/2	Generated stub из .proto	ServerInterceptor

Принципиально, что несмотря на разнородность транспортов, четыре из шести типов (@RestController, Functional, Spring-WS, GraphQL) используют *один и тот же бин IdempotencyFilter*, поскольку все четыре работают поверх Servlet API. gRPC использует протокол HTTP/2 с бинарным protobuf и не проходит через сервлет-цепочку; для него реализован отдельный GrpcIdempotencyInterceptor, делегирующий, однако, *тому же* бину IdempotencyService. Spring Data REST интегрируется через слой данных (@Version и UNIQUE constraints), а не через заголовок.

2.7.1 @RestController (аннотационный подход)

Классический подход, применяемый в Spring Guides [4]. Контроллер аннотируется `@RestController`, маршруты определяются через `@GetMapping`, `@PostMapping` и т. д. Обработка запроса делегируется сервисному слою. Этот подход является наиболее распространённым в экосистеме Spring и обеспечивает декларативное определение API через аннотации.

Архитектура следует паттерну «контроллер–сервис–репозиторий»: контроллер принимает HTTP-запрос, валидирует DTO, передаёт в сервис, который содержит бизнес-логику и взаимодействует с репозиторием для доступа к данным. Идемпотентность POST обеспечивается внешним HTTP-фильтром (`IdempotencyFilter`), который перехватывает запрос до попадания в контроллер.

2.7.2 Functional Endpoints (WebMvc.fn)

Функциональный подход из Spring Framework [5]. Маршруты определяются программно через `RouterFunction<ServerResponse>`, а обработчики — через функции `ServerRequest → ServerResponse`. Полностью отсутствуют аннотации маршрутизации.

Ключевое отличие от `@RestController` — в способе композиции маршрутов. Вместо распределения маршрутов по аннотациям на методах, все маршруты определяются в одном месте (`PaymentRouter`) через цепочку вызовов `.GET()`, `.POST()` и т. д. Обработчики (`PaymentHandler`) — это обычные компоненты Spring, не наследующие от каких-либо контроллерных классов. Такой подход обеспечивает лучшую тестируемость отдельных обработчиков в изоляции.

Идемпотентность POST обеспечивается тем же `IdempotencyFilter`, что и для `@RestController`, поскольку фильтры работают на уровне Servlet API, до маршрутизации запроса.

2.7.3 Spring Data REST

Автоматическая генерация CRUD-эндпоинтов из JPA-репозитория [6]. Контроллер не пишется вручную — достаточно аннотировать репозиторий `@RepositoryRestResource`. Spring Data REST автоматически создаёт все CRUD-операции, поддерживает HATEOAS (Hypermedia As The Engine Of Application State), генерирует HAL-формат ответов и предоставляет HAL Explorer для интерактивного исследования API.

Идемпотентность обеспечивается встроенными механизмами:

- `@Version` (optimistic locking) — Hibernate проверяет версию при каждом UPDATE, Spring Data REST генерирует ETag из значения версии и поддерживает заголовки `If-Match` и `If-None-Match`;

- UNIQUE constraints на уровне базы данных предотвращают создание дубликатов;
- `@RepositoryEventHandler` позволяет добавить бизнес-логику через обработчики событий (`HandleBeforeCreate`, `HandleBeforeSave`).

2.7.4 Spring-WS (SOAP, контракт-первый)

SOAP — транспорт-предшественник REST, до сих пор широко используемый в enterprise и B2B-интеграциях (банковские шлюзы, государственные сервисы). Spring-WS [7] реализует контракт-первый подход: разработчик описывает XSD-схему сообщений, фреймворк автоматически генерирует WSDL и маршрутизирует входящие SOAP-конверты к Java-методам по `namespace+localPart`.

В нашей реализации (домен `shipments`) сервлет `MessageDispatcherServlet` зарегистрирован под URL-паттерном `/api/soap/*`. Контракт описан в `xsd/shipments.xsd` с типами `createShipmentRequest`, `createShipmentResponse` и т.д. Энд-поинт `ShipmentEndpoint` аннотирован `@Endpoint`, методы — `@PayloadRoot(namespace=..., localPart=...)` и `@ResponsePayload`; обработчик принимает `DOMSource` и возвращает `DOMSource` с ответом.

Ключевой момент с точки зрения идемпотентности: `MessageDispatcherServlet` — это обычный `HttpServlet`, поэтому *тот же бин* `IdempotencyFilter` (зарегистрированный в `WebMvcConfig` с URL-паттерном `/api/soap/*`) перехватывает SOAP POST-запросы по заголовку `Idempotency-Key` и кеширует SOAP-envelope ответа целиком. Это работает только при условии, что фильтр транспорт-агностичен — то есть сохраняет реальный `Content-Type` ответа (`text/xml`) и реплеит его без подмены на `application/json`. Соответствующая модификация фильтра описана в § 2.8.

2.7.5 gRPC (HTTP/2 + protobuf)

gRPC [8] принципиально отличается от HTTP/REST: транспорт — HTTP/2 с бинарными protobuf-сообщениями [14], контракт описывается в файле `.proto`, серверные стабы генерируются плагином `protoc-gen-grpc-java`. Сервлет-фильтр здесь *неприменим* — запросы не проходят через `ServletFilterChain`.

В нашей реализации (домен `notifications`) используется стартер `grpc-spring-boot-starter` [15]; gRPC-сервис `NotificationGrpcService` помечен `@GrpcService`. Для идемпотентности написан `GrpcIdempotencyInterceptor` (`@GrpcGlobalServerInterceptor`), который:

1. Извлекает `idempotency-key` из `Metadata` вызова (gRPC-эквивалент HTTP-заголовков, lowercase ASCII).
2. При повторном вызове с тем же ключом достаёт сохранённые байты ответа из `idempotency_keys.response_body_bytes` и парсит их через `call.getMethodDescriptor().getResponseMarshaller().parse(...)`. Это делает перехватчик *полностью generic*: он не знает proto-типы и работает с любым unary-методом любого gRPC-сервиса в приложении.
3. При новом вызове регистрирует ключ через тот же `IdempotencyService`, оборачивает `call` в `ForwardingServerCall.SimpleForwardingServerCall`, перехватывает `sendMessage(RespT)`, сериализует ответ через тот же marshaller (`InputStream.readAllBytes()`) и сохраняет байты в БД (`response_kind=BINARY`).

Таким образом, gRPC-перехватчик переиспользует *тот же бин* `IdempotencyService` и *ту же таблицу* `idempotency_keys`, что и HTTP-фильтр; различается только способ перехвата сообщения и кодировка тела (бинарный protobuf в столбце BYTEA вместо текстового JSON/XML в TEXT).

2.7.6 GraphQL (single endpoint, schema-first)

GraphQL [16] — современный транспорт с одним эндпоинтом (POST /graphql), декларативной схемой типов и клиент-управляемой выборкой полей. В Spring Boot интегрируется через `spring-boot-starter-graphql` [9]: схема описывается в `schema.graphqls`, методы контроллера аннотируются `@QueryMapping` и `@MutationMapping`.

В нашей реализации (домен `inventories`) определены два запроса (`inventories`, `inventory(id)`) и одна мутация (`reserveInventory`). Идем-потентность мутаций обеспечивается *тем же* `IdempotencyFilter`: URL-паттерн `/graphql` добавлен в фильтр, и при мутации (всегда POST) требуется заголовок `Idempotency-Key`. Запросы (Query) могут отправляться через HTTP GET — фильтр пропускает все не-POST, что согласуется с safe-семантикой запросов и снимает требование ключа для них.

Особенность GraphQL: даже при логической ошибке (валидация схемы, отсутствие поля) сервер возвращает HTTP 200 с массивом `errors[]` в JSON-теле. Фильтр кеширует все 2xx-ответы по умолчанию — это соответствует политике IETF Idempotency-Key draft [12]: клиент, повторяющий запрос с тем же ключом, явно хочет тот же *исход*, включая ошибку.

2.8 Транспорт-агностичный снимок ответа

Исходный `IdempotencyFilter` хранил кешированный ответ как пару `<status,body>` и при реплее устанавливал жёстко зафиксированный `Content-Type: application/json`. Это было корректно для исходных JSON-эндпоинтов, но **нарушает транспорт-агностичность**: при добавлении SOAP-эндпоинта реплей вернул бы XML-байты с заголовком `application/json`, что сломало бы парсинг на стороне клиента. Для gRPC задача ещё сложнее: бинарный `protobuf` принципиально не вписывается в TEXT-колонку.

Чтобы сделать механизм идемпотентности по-настоящему транспорт-агностичным, кешированный ответ был доопределён как *полный снимок*:

$$\text{Snapshot} = \langle \text{status}, \text{contentType}, \text{kind}, \text{body} \rangle \quad (7)$$

где `kind` $\in \{\text{TEXT}, \text{BINARY}\}$ — дискриминатор кодировки тела (текст в `response_body` TEXT либо байты в `response_body_bytes` BYTEA), а `contentType` восстанавливается из заголовка фактического ответа.

Изменения, реализованные миграцией V4 и доработкой фильтра:

- Таблица `idempotency_keys` расширена тремя колонками: `response_content_type` VARCHAR(150), `response_kind` VARCHAR(16), `response_body_bytes` BYTEA.
- `IdempotencyService` получил пару методов `saveTextResponse(...)` и `saveBinaryResponse(...)` вместо одного перегруженного.
- `IdempotencyFilter` после `filterChain.doFilter` читает реальный `Content-Type` через `ContentCachingResponseWrapper.getHeader(HttpHeaders.CONTENT_TYPE)` (с fallback на `getContentType()`), а при реплее пишет тело через `response.getOutputStream()` — унифицированно по байтам, без перекодирования через `Writer` (что важно для XML с декларацией кодировки).

После этой модернизации *один и тот же* `IdempotencyFilter` корректно работает на четырёх HTTP-транспортах (REST-JSON, SOAP-XML, GraphQL-JSON), а `IdempotencyService` переиспользуется gRPC-перехватчиком для бинарных ответов. Это и есть main архитектурный вклад данной работы.

2.9 Средства тестирования

- **JUnit 5** — фреймворк модульного тестирования, поддерживающий параметризованные тесты, вложенные классы и расширения [17].

- **Spring Boot Test** — интеграционное тестирование Spring-приложений с автоконфигурацией контекста [18].
- **MockMvc** — тестирование HTTP-запросов без запуска реального HTTP-сервера, с полной поддержкой фильтров и маршрутизации.
- **Testcontainers** — библиотека для запуска Docker-контейнеров в тестах. Используется singleton-паттерн: один контейнер PostgreSQL запускается при старте первого теста и переиспользуется всеми последующими [10].
- **Spring Profiles** — переключение конфигурации для тестирования. Профиль `test` включает идемпотентность, профиль `test-non-idempotent` отключает её, что позволяет запускать один и тот же набор тестов в двух режимах.

3 Результаты

3.1 Описание разработанного программного обеспечения

Разработано Spring Boot приложение, реализующее серверную часть для **шести доменов**, объединённых единым механизмом идемпотентности:

- **Заказы** (`/api/orders`) — аннотационный `@RestController`;
- **Платежи** (`/api/payments`) — функциональные эндпоинты (`RouterFunction`);
- **Товары** (`/api/products`) — автогенерируемые эндпоинты Spring Data REST;
- **Отправления** (`/api/soap/`) — SOAP-эндпоинт на Spring-WS, контракт `xsd/shipments.xsd`;
- **Уведомления** (`localhost:9090`) — gRPC-сервис, контракт `notification.proto`;
- **Инвентарь** (`/graphql`) — GraphQL-контроллер, схема `schema.graphqls`.

Стек технологий: Spring Boot 3.3.6, Java 17, PostgreSQL 16, Flyway, Testcontainers, Docker Compose. Дополнительно: Spring-WS 4.0 + wsdl4j (SOAP), `grpc-spring-boot-starter 3.1` + `grpc-java 1.65` + `protobuf 3.25` (gRPC), `spring-boot-starter-graphql` (GraphQL).

3.1.1 Структура проекта

```
src/main/java/com/example/idempotency/
|-- config/                                # WebMvcConfig (URL-
паттерны фильтра), IdempotencyProperties
|-- idempotency/                          # Filter, Service, Entity, Repository, Res
|-- order/                                # @RestController + Service + Repository
|-- payment/                              # Functional: PaymentRouter + Handler
|-- product/                              # Spring Data REST: ProductRepository + Ev
|-- soap/shipment/                       # SOAP: ShipmentEndpoint + Service + WebSe
|-- grpc/                                  # GrpcIdempotencyInterceptor (общий для вс
сервисов)
| ' -- notification/                      # gRPC-домен: NotificationGrpcService + Se
|-- graphql/inventory/                    # GraphQL: InventoryGraphQLController + Se
+-- exception/                            # Глобальный обработчик ошибок
src/main/proto/notification.proto
src/main/resources/
|-- xsd/shipments.xsd
|-- graphql/schema.graphqls
|-- db/migration/                          # V1..V7
+-- db/migration-non-idempotent/          # V3, V8
```

3.1.2 Схема базы данных

База данных состоит из **семи таблиц**, управляемых миграциями Flyway V1–V7 (плюс V8 в профиле non-idempotent):

Таблица 4: Схема базы данных

Таблица	Ключевые поля	Constraints	Назначение
orders	id (UUID), description, amount, status	UNIQUE(desc, amount)	REST/@RestController
payments	id (UUID), order_id (FK), amount, status	FK → orders	REST/Functional
products	id (BIGSERIAL), sku, name, price, version	UNIQUE(sku)	Spring Data REST
shipments	id (UUID), tracking_number, recipient, status	UNIQUE(tracking_number)	SOAP
notifications	id (UUID), recipient, message, message_hash, status	UNIQUE(recipient, msg_hash)	gRPC
inventories	id (UUID), item_name, quantity	UNIQUE(item_name)	GraphQL
idempotency_keys	id, key, method, path, response_status, response_body, response_content_type, response_kind, response_body_bytes	UNIQUE(key)	Снэпшоты ответов (V4)

Расширение `idempotency_keys` в миграции V4 (§ 2.8) добавляет три колонки, без которых фильтр не мог бы корректно реплеить SOAP/XML и gRPC/protobuf. Поле `response_kind` различает текстовое тело (TEXT, используется JSON/XML) и бинарное (BINARY, используется protobuf); `response_content_type` восстанавливает оригинальный MIME-тип на

реплее. TTL ключей задаётся колонкой `expires_at` (по умолчанию 24 часа).

3.1.3 Компоненты механизма идемпотентности

Механизм идемпотентности реализован через четыре компонента, переиспользуемых четырьмя HTTP-транспортами и одним gRPC-перехватчиком:

1. **IdempotencyFilter** — единственный HTTP-фильтр (наследник `OncePerRequestFilter`), зарегистрированный на URL-паттернах четырёх транспортов: `/api/orders/*`, `/api/payments/*`, `/api/soap/*`, `/graphql`. Реализует алгоритм 1. Отключается параметром `app.idempotency.enabled=false`. После доработки V4 (§ 2.8) транспорт-агностичен.
2. **GrpcIdempotencyInterceptor** — `@GrpcGlobalServerInterceptor`, делегирующий тому же `IdempotencyService`. Generic-реализация работает с любым unary-методом любого gRPC-сервиса через `MethodDescriptor.getResponseMarshaller()`.
3. **IdempotencyService** — единая точка интеграции для всех транспортов. Методы `findByKey()`, `registerKey()`, `saveTextResponse()` (для JSON/XML), `saveBinaryResponse()` (для protobuf-байтов). Операции транзакционны.
4. **IdempotencyKeyEntity** — JPA-сущность, представляющая запись в таблице `idempotency_keys`. Содержит ключ, HTTP-метод, путь запроса, снэпшот ответа (`{status, contentType, kind, body}`) и временные метки.

3.1.4 Профили конфигурации

Приложение поддерживает два режима работы через Spring Profiles:

- **Профиль по умолчанию:** идемпотентность включена (`enabled=true`), Flyway применяет миграции V1 и V2, все constraints активны.
- **Профиль `non-idempotent`:** идемпотентность отключена (`enabled=false`), Flyway дополнительно применяет миграции V3 (дроп UNIQUE на `orders`, `products`) и V8 (дроп UNIQUE на `shipments`, `notifications`, `inventories`). gRPC-перехватчик так же проверяет флаг `enabled` и при выключенном режиме пропускает вызов без перехвата.

Переключение профилей позволяет запускать одни и те же тесты в двух конфигурациях и наглядно демонстрировать последствия отсутствия механизмов идемпотентности на всех шести транспортах.

3.2 Результаты тестирования (профиль idempotent)

Полный набор из 36 интеграционных тестов проходит успешно:

Таблица 5: Результаты тестирования (объединённый прогон обоих профилей)

Тестовый класс	Тестов	Результат	Тип контроллера / транспорт
OrderIdempotencyTest	6	PASS	@RestController (HTTP/JSON)
PaymentIdempotencyTest	5	PASS	Functional Endpoints (HTTP/JSON)
ProductIdempotencyTest	5	PASS	Spring Data REST (HTTP/HAL)
SoapIdempotencyTest	3	PASS	Spring-WS (SOAP/XML)
GrpcIdempotencyTest	4	PASS	@GrpcService (HTTP/2 + protobuf)
GraphQLIdempotencyTest	4	PASS	spring-boot-starter-graphql
DataImportTest	2	PASS	Импорт JSON/CSV
NonIdempotentProfileTest	7	PASS	Демонстрация багов на 6 транспортах
Итого	36	PASS	

3.3 Тест-кейсы по контроллерам

3.3.1 @RestController (OrderIdempotencyTest)

1. **POST с одинаковым Idempotency-Key дважды** — создаётся только один заказ, второй запрос возвращает кэшированный ответ. Проверяется через утверждение `assertEquals(1, count)`.
2. **POST с разными ключами** — создаются два разных заказа, что подтверждает: идемпотентность привязана к конкретному ключу, а не к содержимому запроса.
3. **POST без Idempotency-Key** — возвращается 400 Bad Request, что предотвращает случайное создание неидемпотентных запросов.
4. **PUT идемпотентен** — повторное обновление с теми же данными не изменяет состояние. Проверяется сравнением полей `description`, `amount`, `status` после двух последовательных PUT.
5. **DELETE идемпотентен** — первый вызов возвращает 204 No Content (ресурс удалён), второй — 404 Not Found (ресурс уже отсутствует). Оба варианта соответствуют идемпотентному поведению.

6. **GET идиempотентен** — три последовательных GET-запроса возвращают одинаковый JSON-ответ.

3.3.2 Functional Endpoints (PaymentIdempotencyTest)

1. **POST платежа с одним ключом дважды** — создаётся один платёж. Демонстрирует, что IdempotencyFilter работает с RouterFunction так же, как с @RestController.
2. **POST с разными ключами** — создаются два платежа.
3. **POST без ключа** — 400 Bad Request.
4. **GET идиempотентен** — стабильный результат при многократных запросах.
5. **Retry (3 запроса с одним ключом)** — создаётся один платёж, несмотря на 3 попытки. Это ключевой production-сценарий: клиент повторяет запрос из-за таймаута, но система обрабатывает его только один раз.

3.3.3 Spring Data REST (ProductIdempotencyTest)

1. **POST дубликата SKU** — 409 Conflict. UNIQUE constraint на поле sku предотвращает создание товара с тем же артикулом.
2. **PUT с корректной версией** — успешное обновление, @Version автоматически инкрементируется. Проверяется совпадение version + 1 с актуальной версией.
3. **Concurrent PUT (два обновления со старой версией)** — первое обновление проходит успешно (версия 0 → 1), второе получает 412 Precondition Failed, поскольку передан устаревший ETag. Optimistic locking предотвращает lost update.
4. **PATCH идиempотентен** — повторное применение одного и того же PATCH не изменяет данные (имя товара остаётся тем же после второго PATCH).
5. **GET идиempотентен** — стабильный результат.

3.3.4 Импорт данных (DataImportTest)

1. **Импорт заказов из JSON** — загрузка данных из файла orders-input.json, создание через API, сравнение количества, статусов и сумм с эталонным файлом.

2. **Импорт товаров из CSV** — загрузка данных из файла `products-input.csv` через Jackson CSV, сохранение в БД, затем повторная попытка импорта: все 3 попытки отклоняются `UNIQUE constraint`, количество записей не увеличивается.

3.3.5 SOAP (SoapIdempotencyTest)

Тесты используют `TestRestTemplate` на random-порту (`MockMvc` маршрутизирует только в `DispatcherServlet` и не вызывает `MessageDispatcherServlet`), отправляя сырой SOAP-envelope в `/api/soap/`:

1. **Один Idempotency-Key дважды** — ответы байт-в-байт идентичны (включая `Content-Type text/xml`), в `shipments` ровно одна запись. Это подтверждает, что доработанный фильтр (§ 2.8) корректно реплеит XML-тело.
2. **Разные ключи** — две записи `shipments`.
3. **POST без Idempotency-Key** — 400 Bad Request, ничего не создано.

3.3.6 gRPC (GrpcIdempotencyTest)

Тесты подключаются через `InProcessChannelBuilder` к in-process gRPC-серверу (`grpc.server.in-process-name=idempotency-test`); идемпотентный ключ передаётся в `Metadata`:

1. **Send с одним ключом дважды** — `SendResponse.id` совпадает, в `notifications` одна запись. Реплей происходит через десериализацию байтов из `response_body_bytes` тем же `marshaller`, что использовался при сериализации.
2. **Разные ключи** — две записи `notifications`.
3. **Без idempotency-key** — `Status.INVALID_ARGUMENT`, ничего не создано.
4. **Бинарная идентичность реплея** — `toByteArray()` обоих ответов совпадают побайтно. Это сильнее, чем равенство объектов: подтверждает, что сериализованная форма стабильна.

3.3.7 GraphQL (GraphQLIdempotencyTest)

Тесты используют `TestRestTemplate` на `/graphql`, отправляя GraphQL-запросы JSON-обёрткой:

1. **Мутация с одним Idempotency-Key дважды** — JSON-ответы идентичны, в `inventories` одна запись.
2. **Мутации с разными ключами** — две записи.
3. **Мутация без ключа** — 400 Bad Request.
4. **Query после мутации** — запрос возвращает только что созданную запись, что демонстрирует совместимость GraphQL Query с механизмом идемпотентности (Query через POST требует ключ, но это не мешает выборке данных).

3.4 Результаты профиля non-idempotent

Профиль `non-idempotent` отключает механизмы идемпотентности на всех слоях:

- `IdempotencyFilter` пропускает все POST-запросы без проверки заголовка (`enabled=false`);
- `GrpcIdempotencyInterceptor` обходит логику регистрации ключа по тому же флагу;
- Миграция V3 удаляет UNIQUE constraints на `orders` и `products`;
- Миграция V8 удаляет UNIQUE constraints на `shipments`, `notifications`, `inventories`.

Таблица 6: Демонстрация проблем без идемпотентности (профиль `non-idempotent`, все 6 транспортов)

Тест	Обнаруженная проблема	Статус
REST POST без Idempotency-Key	Запрос проходит без проверки — любой POST создаёт ресурс	BUG
REST повторный POST с тем же ключом	Создаётся дубликат заказа — ключ игнорируется	BUG
Data REST дубликат SKU	Создаётся второй товар с тем же артикулом	BUG
Functional retry платежа (3 раза)	3 платежа вместо 1 — тройное списание средств	BUG
SOAP createShipment с тем же tracking_number	Создаются два Shipment с одинаковым номером	BUG
gRPC Send с тем же idempotency-key	Interceptor выключен ⇒ дубликат уведомления	BUG
GraphQL мутация с тем же ключом	Фильтр выключен ⇒ два Inventory с одним item_name	BUG

Все семь тестов `NonIdempotentProfileTest` проходят зелёными, подтверждая наличие проблем во всех шести транспортах. Каждый тест использует прямые assertions: например, проверяется, что `notificationRepository.count() == 2` — без перехватчика gRPC и без DB constraint действительно создаются дубликаты. Это демонстрирует критическое значение комбинации *транспортного механизма* (фильтр/перехватчик) и *domain-уровня* (UNIQUE constraint) для корректного функционирования системы независимо от выбранного протокола.

4 Обсуждение

4.1 Сравнение подходов к обеспечению идемпотентности

Результаты исследования показывают, что разные транспорты требуют различных подходов к обеспечению идемпотентности, однако *бизнес-логика идемпотентности* (`IdempotencyService`) единая на все шесть типов контроллеров (таблица 7).

Таблица 7: Сравнение механизмов идемпотентности по типам контроллеров и транспортам

Тип / транспорт	Механизм перехвата	Хранение снимка	Domain-уровень защиты
@RestController	IdempotencyFilter (servlet)	TEXT (JSON)	UNIQUE(desc, amount)
Functional Endpoints	<i>тот же</i> фильтр	TEXT (JSON)	—
Spring Data REST	@Version + ETag	—	UNIQUE(sku)
Spring-WS @Endpoint	<i>тот же</i> фильтр	TEXT (XML, text/xml)	UNIQUE(tracking_number)
@GrpcService	GrpcIdempotencyInterceptor	BYTEA (protobuf)	UNIQUE(recipient, msg_hash)
GraphQL @Controller	<i>тот же</i> фильтр	TEXT (JSON)	UNIQUE(item_name)

Ключевое наблюдение: **четыре из шести транспортов используют один и тот же бин `IdempotencyFilter`** (отличаясь только URL-паттерном), пятый (gRPC) переиспользует тот же `IdempotencyService` через тонкий перехватчик, а шестой (Spring Data REST) интегрируется через слой данных. Это иллюстрирует архитектурный принцип ортогональности: транспорт и идемпотентность — независимые измерения, и общий механизм может быть применён к любому транспорту, поддерживающему перехват запроса/ответа.

4.2 Анализ результатов по транспортам

Аннотационный @RestController предоставляет наибольшую гибкость в реализации идемпотентности, но требует ручной реализации фильтра и хранилища ключей. Этот подход наиболее распространён в production-системах благодаря явности и читаемости кода. Разработчик

полностью контролирует обработку запроса, может добавить произвольную логику валидации, трансформации и обработки ошибок. Недостаток — необходимость писать и поддерживать больше кода.

Функциональные эндпоинты (`WebMvc.fn`) используют тот же механизм `Idempotency-Key` через общий фильтр, что демонстрирует важное архитектурное преимущество — фильтры работают на уровне `Servlet API`, независимо от типа контроллера. Это означает, что один и тот же `IdempotencyFilter` обеспечивает идемпотентность как для `@RestController`, так и для `RouterFunction`. Функциональный стиль обеспечивает лучшую композиционность: маршруты можно комбинировать, вкладывать и переиспользовать программно, в отличие от аннотационного подхода, где маршруты привязаны к методам классов.

Spring Data REST автоматически обеспечивает ряд механизмов идемпотентности: `@Version` генерирует `ETag`, а `unique constraints` предотвращают дубликаты. Это минимизирует объём ручного кода — разработчику достаточно определить JPA-сущность и репозиторий. Однако этот подход ограничивает возможности кастомизации: `Idempotency-Key` не поддерживается автоматически, а бизнес-логика добавляется через `EventHandler`, что менее гибко, чем прямое управление в `Service`-слое.

Spring-WS (SOAP) был самым простым в интеграции с существующим фильтром: `MessageDispatcherServlet` — обычный `HttpServlet`, поэтому добавление URL-паттерна `/api/soap/*` в `FilterRegistrationBean` автоматически включило идемпотентность для всех SOAP-вызовов. Единственное препятствие — описанная в § 2.8 проблема с `hardcoded Content-Type`: без сохранения реального типа фильтр вернул бы XML-байты с заголовком `JSON`. SOAP примечателен ещё тем, что контракт-первый подход (`XSD` → `WSDL`) автоматически документирует API; недостаток — многословность XML и накладные расходы на парсинг по сравнению с `JSON`.

gRPC потребовал наибольших архитектурных усилий: пришлось написать отдельный `ServerInterceptor`, ввести бинарную ветку в схему `idempotency_keys` (`BYTEA` вместо `TEXT`) и решить, как реплеить ответ без знания конкретного прото-типа. Решение через `MethodDescriptor.getResponseMarshaller()` оказалось элегантным: перехватчик generic и работает с любым unary-методом любого `@GrpcService` в приложении. Преимущества gRPC ярко проявляются в performance-чувствительных сценариях: бинарный `protobuf` компактнее `JSON` в 2–4 раза, а `HTTP/2`-мультиплексирование снимает `overhead` на установку соединений. Недостаток — меньшая совместимость с инфраструктурой (нет браузерной поддержки без `gRPC-Web`, сложнее отладка через `curl`).

GraphQL использовал тот же фильтр, что и REST, — `spring-boot-starter-graphql` регистрирует один `HTTP-endpoint`, на который и навешивается фильтр. Особенность GraphQL — семантически перекрёстная природа: мутации требуют ключа, запросы могут идти через `GET` без ключа.

В нашей реализации фильтр пропускает не-POST автоматически, что естественно соответствует safe-семантике Query. Дискуссионный момент — кеширование ошибок: GraphQL возвращает HTTP 200 даже при логических ошибках в `errors[]`; принятая в работе политика «кешировать все 2xx» соответствует букве IETF Idempotency-Key draft, но в production-системах может потребоваться более тонкая логика.

4.3 Идемпотентность как конвенция: почему нельзя «всё делать через GET»

Изложенные выше свойства HTTP-методов (таблица 1) могут привести на естественный вопрос: если GET и HEAD одновременно безопасны (safe) и идемпотентны, почему бы не использовать их для всех операций? Ответ принципиально важен для понимания природы идемпотентности: **свойства HTTP-методов — это семантическое соглашение, а не техническое ограничение, накладываемое протоколом.** RFC 9110 [3] определяет ожидаемое поведение методов, но сам по себе протокол не запрещает серверу изменить состояние в ответ на GET или проигнорировать идемпотентность PUT. Иными словами, *фактическая* идемпотентность и безопасность целиком определяются реализацией на стороне сервера. Тем не менее, отступление от конвенции влечёт за собой ряд серьёзных последствий.

- **Кэширование на промежуточных узлах.** Ответы на GET по умолчанию кэшируются браузерами, прямыми и обратными прокси, CDN, service worker'ами. Если GET меняет состояние, второй запрос клиента может быть обслужен из кэша и вообще не дойти до сервера — операция «пропадёт». Для POST и PUT такого кэширования по умолчанию не происходит.
- **Предварительная выборка и префетчинг.** Браузеры, поисковые роботы, антивирусные сканеры ссылок и системы link preview (например, в мессенджерах) свободно вызывают GET на любых обнаруженных URL. Если такой запрос порождает побочный эффект, возможны спонтанные изменения данных без ведома пользователя. Исторический пример — эпизод с Google Web Accelerator, который проходил по ссылкам «Удалить», реализованным через GET, и удалял записи пользователей.
- **Утечка данных в URL.** Параметры GET передаются в строке запроса и попадают в журналы веб-серверов, историю браузера, заголовки Referer, аналитические системы. Это делает GET непригодным для передачи чувствительных данных (паролей, токенов, платёжных реквизитов).

- **Тело запроса.** Согласно RFC 9110, GET формально может содержать тело, но его семантика не определена, и многие прокси, CDN и HTTP-клиенты тело у GET удаляют. HEAD тела ответа вообще не возвращает и пригоден только для проверки метаданных ресурса.
- **Повторные попытки и отказоустойчивость.** HTTP-клиенты, балансировщики и сервис-мешы автоматически повторяют только те методы, которые спецификация объявляет идемпотентными. Если GET имеет побочный эффект, такой автоматический retry приведёт к множественному выполнению операции.
- **Наблюдаемость и проверяемость контракта.** API-шлюзы, системы мониторинга, тесты на соответствие REST-конвенциям (например, линтеры OpenAPI) исходят из стандартной семантики методов. Нарушение этой семантики делает систему непредсказуемой для сторонних интеграторов.

Таким образом, общепринятое соответствие между методом и типом операции — GET для чтения, POST для создания, PUT для полной замены, PATCH для частичного обновления, DELETE для удаления — является не техническим, а **инженерным требованием**, вытекающим из принципа наименьшего удивления [19]. Разработчик обязан реализовать эти свойства самостоятельно: GET-эндпоинт должен быть read-only, а не «кажущийся» read-only. В этом смысле тестирование идемпотентности — это проверка не протокола, а соответствия реализации декларируемому контракту. Настоящая работа построена именно на этом принципе: для каждого метода проверяется не формальная корректность HTTP, а фактическая инвариантность состояния системы при повторных вызовах.

4.4 Оптимистичная блокировка и границы её применимости для идемпотентности

Механизм @Version, продемонстрированный на примере Spring Data REST для сущности Product, часто преподносится как «решение проблемы идемпотентности». Это упрощение требует уточнения: **версионирование в общем случае не обеспечивает идемпотентность; оно решает связанную, но иную задачу — предотвращение lost update при конкурентных обновлениях** [13; 20].

Различие становится очевидным при рассмотрении повторных запросов. Пусть клиент отправляет PUT /products/42 с заголовком If-Match: "3". Первый запрос успешно меняет версию $v = 3 \rightarrow v = 4$ и возвращает 200 OK. Сетевой таймаут приводит к повторной отправке того же запроса с тем же If-Match: "3". Второй запрос получает 412 Precondition

Failed, поскольку текущая версия уже равна 4. С точки зрения строгого определения идемпотентности это *нарушение*: два одинаковых запроса дали разные результаты (200 и 412), несмотря на то что итоговое состояние ресурса идентично. Клиент, получивший 412, вынужден либо интерпретировать этот код как «успех, просто повторно», либо выполнить дополнительный **GET** для перечитывания версии — что усложняет протокол взаимодействия.

Идемпотентность **PUT** в нашей системе обеспечивается не аннотацией **@Version**, а самой семантикой метода **PUT** «полная замена состояния»: повторная отправка одинакового тела без **If-Match** приводит к одному и тому же финальному состоянию ресурса. **@Version** же решает *ортогональную* задачу — разруливание конфликта между двумя разными клиентами, одновременно редактирующими один ресурс на основе устаревшей версии.

Частный случай, в котором версионирование действительно совпадает с идемпотентностью, — это **PUT**-операции, где запрос содержит ожидаемую версию не в заголовке **If-Match**, а в теле запроса как часть бизнес-смысла (например, «применить эту дельту к версии 3»). В такой модели повторный запрос с **expectedVersion = 3** после успеха первого вернёт тот же самый результат «ожидаемая версия устарела» для обоих повторов, что формально идемпотентно. Именно так работают многие системы *event sourcing* и *CQRS*. В нашей работе подобная схема *не* используется: **@Version** на **Product** приведён исключительно как демонстрация того, что **Spring Data REST** предоставляет **optimistic locking** «из коробки» через заголовки **ETag/If-Match**, чтобы показать разнообразие механизмов, которые *Spring*-экосистема относит к тематике «идемпотентности и повторяемости запросов». В *production*-системе, где действительно требуется идемпотентность **PUT** при *retry*, следует применять не **@Version**, а паттерн **Idempotency-Key** [2; 12] — именно он сохраняет *ответ* первого выполнения и возвращает его без повторного исполнения операции.

Таким образом, **@Version** и **Idempotency-Key** — дополняющие, а не взаимозаменяемые механизмы: первый защищает от *двух разных* запросов от разных клиентов, второй — от *одного и того же* запроса, повторённого из-за сбоя сети.

4.5 Значимость Testcontainers в тестировании идемпотентности

Использование **Testcontainers** вместо *in-memory* баз данных (**H2**) имеет критическое значение для тестирования идемпотентности. Ряд механизмов зависит от особенностей конкретной СУБД:

- **UNIQUE constraints** — поведение при нарушении уникальности различается между **H2** и **PostgreSQL** (разные коды ошибок, разная

обработка транзакций).

- **Flyway-миграции** — синтаксис SQL может отличаться (например, `gen_random_uuid()` специфичен для PostgreSQL).
- **Конкурентный доступ** — PostgreSQL поддерживает уровни изоляции и блокировки, отличные от H2.

Singleton-паттерн Testcontainers (один контейнер на все тесты) обеспечивает баланс между точностью и скоростью: тесты работают с реальной СУБД, но контейнер запускается только один раз за сессию.

4.6 Рекомендации для production

На основе проведённого исследования сформулированы следующие рекомендации:

1. **Всегда требовать Idempotency-Key для POST-запросов**, создающих ресурсы или выполняющих побочные эффекты. Это предотвращает дублирование при retry и является отраслевым стандартом [2].
2. **Использовать optimistic locking (@Version)** для защиты от конкурентных обновлений. Это особенно важно для систем с высокой нагрузкой.
3. **Устанавливать TTL** для ключей идемпотентности. Рекомендуемое значение — 24 часа [12]. Слишком короткий TTL не защитит от поздних retry, слишком длинный — приведёт к росту таблицы ключей.
4. **Хранить ключи** в той же СУБД, что и бизнес-данные, для обеспечения транзакционной консистентности. Альтернатива (Redis) обеспечивает лучшую производительность, но вводит риск рассогласования.
5. **Тестировать с реальной СУБД** через Testcontainers [10], а не с in-memory заменителями, для достоверности результатов.
6. **Комбинировать механизмы**: Idempotency-Key для POST, @Version для PUT, UNIQUE constraints как последний рубеж защиты от дубликатов.

5 Заключение

В данной работе проведено исследование и демонстрация тестирования идемпотентности и повторяемости запросов на базе Spring Boot. Разработано приложение с **шестью кардинально различными типами контроллеров**, охватывающими четыре транспортных протокола (HTTP/REST в трёх стилях, SOAP, gRPC, GraphQL), для каждого из которых реализованы и протестированы механизмы обеспечения идемпотентности.

Основные результаты:

- Формализована математическая модель идемпотентности HTTP-запросов с доказательством корректности паттерна Idempotency-Key.
- Реализован и протестирован транспорт-агностичный паттерн Idempotency-Key с хранением ключей и снимков ответов в PostgreSQL. Снимок доопределён как $\langle \text{status}, \text{contentType}, \text{kind}, \text{body} \rangle$, что позволяет корректно реплеить ответы любого формата — JSON, XML/SOAP, бинарный protobuf.
- Продемонстрированы шесть подходов к организации серверных контроллеров в Spring-экосистеме: аннотационный `@RestController`, функциональный `RouterFunction`, автогенерируемый Spring Data REST, контракт-первый `@Endpoint` (Spring-WS, SOAP), `@GrpcService` (gRPC, HTTP/2 + protobuf) и schema-first `@QueryMapping/@MutationMapping` (GraphQL).
- Показано переиспользование единого `IdempotencyService`: четыре HTTP-транспорта работают через *один и тот же бин* `IdempotencyFilter` (различаются только URL-паттерны), gRPC — через тонкий `GrpcIdempotencyInterceptor`, делегирующий тому же сервису через `transport-generic MethodDescriptor.getResponseMarshaller()`.
- Разработано 36 интеграционных тестов с использованием Testcontainers и PostgreSQL, покрывающих все аспекты идемпотентности для шести типов контроллеров.
- Продемонстрировано критическое значение механизмов идемпотентности на всех транспортах: при их отключении (профиль `non-idempotent`) система допускает дублирование данных и тройное списание при retry, что подтверждено отдельным тестом на каждый из шести типов контроллеров.

6 Инструкция по установке и запуску

6.1 Требования

- Java 17+ (JDK)
- Maven 3.8+
- Docker (для PostgreSQL и Testcontainers)

6.2 Запуск приложения

1. Запустить PostgreSQL через Docker Compose:

```
docker-compose up -d
```

2. Собрать и запустить приложение:

```
mvn spring-boot:run
```

3. API доступно по адресу `http://localhost:8080`:

- `/api/orders` — заказы (@RestController)
- `/api/payments` — платежи (Functional Endpoints)
- `/api/products` — товары (Spring Data REST)
- `/api/explorer` — HAL Explorer (Spring Data REST)
- `/api/soap/` — отправления (SOAP via Spring-WS, WSDL по `/api/soap/shipments.wsdl`)
- `/graphql` — инвентарь (GraphQL via spring-boot-starter-graphql)

4. gRPC-сервис уведомлений (NotificationService) поднимается на отдельном порту `localhost:9090` (HTTP/2, plaintext).

6.3 Запуск тестов

1. Запуск всех тестов (профиль с идемпотентностью):

```
mvn test
```

2. Запуск тестов без идемпотентности (демонстрация багов):

```
mvn test -Dtest="NonIdempotentProfileTest"
```

Примечание: для запуска тестов требуется Docker, так как Testcontainers автоматически запускает PostgreSQL в контейнере.

6.4 Пример использования API (все четыре транспорта)

Создание заказа через REST (@RestController) с Idempotency-Key:

```
curl -X POST http://localhost:8080/api/orders \
  -H "Content-Type: application/json" \
  -H "Idempotency-Key: unique-key-123" \
  -d '{"description": "Ноутбук", "amount": 89999.99}'
```

Создание отправления через SOAP (Spring-WS):

```
curl -X POST http://localhost:8080/api/soap/ \
  -H "Content-Type: text/xml" \
  -H "Idempotency-Key: soap-key-1" \
  --data-binary @create-shipment.xml
```

Отправка уведомления через gRPC (grpcurl):

```
grpcurl -plaintext -H "idempotency-key: grpc-key-1" \
  -d '{"recipient":"user@example.com","message":"Hello"}' \
  localhost:9090 \
  com.example.idempotency.grpc.notification.NotificationService/Send
```

Резервирование инвентаря через GraphQL:

```
curl -X POST http://localhost:8080/graphql \
  -H "Content-Type: application/json" \
  -H "Idempotency-Key: gql-key-1" \
  -d '{"query":"mutation { reserveInventory(input:{itemName:\"X\", quan
```

Во всех четырёх случаях повторный запрос с тем же ключом возвращает байт-в-байт идентичный ответ (для gRPC — идентичные сериализованные protobuf-байты), а в базе данных создаётся ровно одна запись.

Список литературы

1. *Helland P.* Idempotence Is Not a Medical Condition // Communications of the ACM. Т. 55. — ACM, 2012. — С. 56—65.
2. *Stripe.* Idempotent Requests — Stripe API. — 2024. — URL: https://stripe.com/docs/api/idempotent_requests ; Stripe API Documentation.
3. *Fielding R., Nottingham M., Reschke J.* HTTP Semantics. — 2022. — URL: <https://www.rfc-editor.org/rfc/rfc9110> ; Internet Engineering Task Force. RFC 9110.
4. *Spring Team.* Building REST services with Spring. — 2024. — URL: <https://spring.io/guides/tutorials/rest> ; Spring Guides.
5. *Spring Framework.* Functional Endpoints — Spring Web MVC. — 2024. — URL: <https://docs.spring.io/spring-framework/reference/web/webmvc-functional.html> ; Spring Framework Reference Documentation.
6. *Spring Team.* Accessing JPA Data with REST. — 2024. — URL: <https://spring.io/guides/gs/accessing-data-rest> ; Spring Guides.
7. *Spring Team.* Spring Web Services — Reference Documentation. — 2024. — URL: <https://docs.spring.io/spring-ws/docs/current/reference/html/> ; Spring Web Services 4.0.
8. *The gRPC Authors.* gRPC Java — Quick Start. — 2024. — URL: <https://grpc.io/docs/languages/java/quickstart/> ; Документация gRPC Java.
9. *Spring Team.* Spring for GraphQL — Reference Documentation. — 2024. — URL: <https://docs.spring.io/spring-graphql/reference/> ; Spring GraphQL 1.3.
10. *AtomicJar.* Testcontainers — Integration testing with real dependencies. — 2024. — URL: <https://testcontainers.com/> ; Официальная документация.
11. *Lang S.* Algebra. Т. 211. — 3-е изд. — Springer, 2002. — (Graduate Texts in Mathematics). — ISBN 978-0387953854.
12. *Dalal S., Jena J.* The Idempotency-Key HTTP Header Field. — 2024. — URL: <https://datatracker.ietf.org/doc/draft-ietf-httpapi-idempotency-key-header/> ; draft-ietf-httpapi-idempotency-key-header-06. IETF Internet-Draft.
13. *Fowler M.* Patterns of Enterprise Application Architecture. — Addison-Wesley, 2002. — ISBN 978-0321127426.

14. *Google*. Protocol Buffers Language Guide (proto3). — 2024. — URL: <https://protobuf.dev/programming-guides/proto3/> ; Документация Protocol Buffers.
15. *Zhang M.* grpc-spring-boot-starter. — 2024. — URL: <https://github.com/yidongnan/grpc-spring-boot-starter> ; Стартер net.devh для интеграции gRPC и Spring Boot.
16. *GraphQL Foundation*. GraphQL Specification (October 2021). — 2021. — URL: <https://spec.graphql.org/October2021/> ; GraphQL Working Group.
17. *Beck K.* Test Driven Development: By Example. — Addison-Wesley, 2002. — ISBN 978-0321146533.
18. *Spring Team*. Testing in Spring Boot. — 2024. — URL: <https://docs.spring.io/spring-boot/reference/testing/index.html> ; Spring Boot Reference Documentation.
19. *Richardson L., Amundsen M., Ruby S.* RESTful Web APIs. — O'Reilly Media, 2013. — ISBN 978-1449358068.
20. *Kleppmann M.* Designing Data-Intensive Applications. — O'Reilly Media, 2017. — ISBN 978-1449373320.

Приложение А. Листинги исходного кода

А.1. docker-compose.yml

Листинг 1: docker-compose.yml — PostgreSQL для разработки

```
1 services:
2   postgres:
3     image: postgres:16-alpine
4     container_name: idempotency-postgres
5     environment:
6       POSTGRES_DB: idempotency_demo
7       POSTGRES_USER: demo
8       POSTGRES_PASSWORD: demo
9     ports:
10      - "5432:5432"
11     volumes:
12      - pgdata:/var/lib/postgresql/data
13     healthcheck:
14       test: ["CMD-SHELL", "pg_isready -U demo -d
15         idempotency_demo"]
16       interval: 5s
17       timeout: 3s
18       retries: 5
19 volumes:
20   pgdata:
```

А.2. application.yml

Листинг 2: application.yml — основная конфигурация

```
1 spring:
2   application:
3     name: idempotency-demo
4   datasource:
5     url: jdbc:postgresql://localhost:5432/idempotency_demo
6     username: demo
7     password: demo
8     driver-class-name: org.postgresql.Driver
9   jpa:
10     hibernate:
11       ddl-auto: validate
12     open-in-view: false
13     properties:
14       hibernate:
15         format_sql: true
16   flyway:
17     enabled: true
18     locations: classpath:db/migration
```

```

19 data:
20   rest:
21     base-path: /api
22
23 app:
24   idempotency:
25     enabled: true
26     key-ttl-hours: 24
27
28 server:
29   port: 8080

```

A.3. application-non-idempotent.yml

Листинг 3: application-non-idempotent.yml — профиль без идемпотентности

```

1 app:
2   idempotency:
3     enabled: false
4
5 spring:
6   flyway:
7     locations: classpath:db/migration,classpath:db/migration-
8               non-idempotent
9   jpa:
10    properties:
11      hibernate:
12        '[jakarta.persistence.lock.timeout]': 0

```

A.4. V1__init__schema.sql

Листинг 4: V1__init__schema.sql — основная схема БД

```

1 -- Таблица заказов (домен OrderController)
2 CREATE TABLE orders (
3   id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
4   description VARCHAR(255) NOT NULL,
5   amount DECIMAL(19, 2) NOT NULL,
6   status VARCHAR(50) NOT NULL DEFAULT 'CREATED',
7   created_at TIMESTAMP NOT NULL DEFAULT NOW(),
8   CONSTRAINT uk_order_description_amount UNIQUE (description,
9   amount)
10 );
11
12 -- Таблица платежей (домен PaymentRouter)
13 CREATE TABLE payments (
14   id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
15   order_id UUID NOT NULL REFERENCES orders(id),
16   amount DECIMAL(19, 2) NOT NULL,

```

```

16     status VARCHAR(50) NOT NULL DEFAULT 'PENDING',
17     created_at TIMESTAMP NOT NULL DEFAULT NOW()
18 );
19
20 -- Таблица товаров (домен ProductRepository)
21 CREATE TABLE products (
22     id BIGSERIAL PRIMARY KEY,
23     sku VARCHAR(100) NOT NULL UNIQUE,
24     name VARCHAR(255) NOT NULL,
25     price DECIMAL(19, 2) NOT NULL,
26     quantity INTEGER NOT NULL DEFAULT 0,
27     version INTEGER NOT NULL DEFAULT 0
28 );

```

A.5. V2__idempotency_keys.sql

Листинг 5: V2__idempotency_keys.sql — таблица ключей идиempотентности

```

1 CREATE TABLE idempotency_keys (
2     id BIGSERIAL PRIMARY KEY,
3     idempotency_key VARCHAR(255) NOT NULL UNIQUE,
4     method VARCHAR(10) NOT NULL,
5     path VARCHAR(500) NOT NULL,
6     response_status INTEGER,
7     response_body TEXT,
8     created_at TIMESTAMP NOT NULL DEFAULT NOW(),
9     expires_at TIMESTAMP NOT NULL DEFAULT NOW() + INTERVAL '24
        hours'
10 );
11
12 CREATE INDEX idx_idempotency_key ON idempotency_keys(
    idempotency_key);
13 CREATE INDEX idx_idempotency_expires ON idempotency_keys(
    expires_at);

```

A.6. V3__drop_constraints.sql

Листинг 6: V3__drop_constraints.sql — удаление ограничений (профиль non-idempotent)

```

1 ALTER TABLE orders DROP CONSTRAINT IF EXISTS
    uk_order_description_amount;
2 ALTER TABLE products DROP CONSTRAINT IF EXISTS products_sku_key
    ;

```

A.7. IdempotencyApplication.java

Листинг 7: IdempotencyApplication.java — точка входа

```
1 package com.example.idempotency;
2
3 import org.springframework.boot.SpringApplication;
4 import org.springframework.boot.autoconfigure.
    SpringApplication;
5
6 @SpringBootApplication
7 public class IdempotencyApplication {
8     public static void main(String[] args) {
9         SpringApplication.run(IdempotencyApplication.class,
10                                args);
11     }
12 }
```

A.8. IdempotencyProperties.java

Листинг 8: IdempotencyProperties.java — настройки идемпотентности

```
1 package com.example.idempotency.config;
2
3 import org.springframework.boot.context.properties.
    ConfigurationProperties;
4
5 @ConfigurationProperties(prefix = "app.idempotency")
6 public class IdempotencyProperties {
7     private boolean enabled = true;
8     private int keyTtlHours = 24;
9
10    public boolean isEnabled() { return enabled; }
11    public void setEnabled(boolean enabled) { this.enabled =
        enabled; }
12    public int getKeyTtlHours() { return keyTtlHours; }
13    public void setKeyTtlHours(int keyTtlHours) { this.
        keyTtlHours = keyTtlHours; }
14 }
```

A.9. WebMvcConfig.java

Листинг 9: WebMvcConfig.java — регистрация фильтра

```
1 package com.example.idempotency.config;
2
3 import com.example.idempotency.idempotency.IdempotencyFilter;
4 import com.example.idempotency.idempotency.IdempotencyService;
5 import org.springframework.boot.context.properties.
    EnableConfigurationProperties;
6 import org.springframework.boot.web.servlet.
    FilterRegistrationBean;
```

```

7 import org.springframework.context.annotation.Bean;
8 import org.springframework.context.annotation.Configuration;
9
10 @Configuration
11 @EnableConfigurationProperties(IdempotencyProperties.class)
12 public class WebMvcConfig {
13     @Bean
14     public FilterRegistrationBean<IdempotencyFilter>
15         idempotencyFilterRegistration(
16             IdempotencyService idempotencyService,
17             IdempotencyProperties properties) {
18         var reg = new FilterRegistrationBean<IdempotencyFilter>
19             >();
20         reg.setFilter(new IdempotencyFilter(idempotencyService,
21             properties));
22         reg.addUrlPatterns("/api/orders/*", "/api/orders",
23             "/api/payments/*", "/api/payments");
24         reg.setOrder(1);
25         return reg;
26     }
27 }

```

A.10. IdempotencyKeyEntity.java

Листинг 10: IdempotencyKeyEntity.java — сущность ключа идемпотентности

```

1 package com.example.idempotency.idempotency;
2
3 import jakarta.persistence.*;
4 import java.time.LocalDateTime;
5
6 @Entity
7 @Table(name = "idempotency_keys")
8 public class IdempotencyKeyEntity {
9     @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
10     private Long id;
11
12     @Column(name = "idempotency_key", nullable = false, unique
13         = true)
14     private String idempotencyKey;
15
16     @Column(nullable = false, length = 10)
17     private String method;
18
19     @Column(nullable = false, length = 500)
20     private String path;
21
22     @Column(name = "response_status")
23     private Integer responseStatus;
24 }

```

```

23
24 @Column(name = "response_body", columnDefinition = "TEXT")
25 private String responseBody;
26
27 @Column(name = "created_at", nullable = false, updatable =
28         false)
29 private LocalDateTime createdAt = LocalDateTime.now();
30
31 @Column(name = "expires_at", nullable = false)
32 private LocalDateTime expiresAt = LocalDateTime.now().
33         plusHours(24);
34
35 public IdempotencyKeyEntity() {}
36 public IdempotencyKeyEntity(String key, String method,
37         String path) {
38         this.idempotencyKey = key; this.method = method; this.
39         path = path;
40 }
41
42 // Getters u Setters
43 public Long getId() { return id; }
44 public String getIdempotencyKey() { return idempotencyKey;
45 }
46 public Integer getResponseStatus() { return responseStatus;
47 }
48 public void setResponseStatus(Integer s) { this.
49         responseStatus = s; }
50 public String getResponseBody() { return responseBody; }
51 public void setResponseBody(String b) { this.responseBody =
52         b; }
53 }

```

A.11. IdempotencyKeyRepository.java

Листинг 11: IdempotencyKeyRepository.java

```

1 package com.example.idempotency.idempotency;
2
3 import org.springframework.data.jpa.repository.JpaRepository;
4 import java.util.Optional;
5
6 public interface IdempotencyKeyRepository
7         extends JpaRepository<IdempotencyKeyEntity, Long> {
8         Optional<IdempotencyKeyEntity> findByIdempotencyKey(String
9                 key);
10 }

```

A.12. IdempotencyService.java

Листинг 12: IdempotencyService.java — сервис управления ключами

```
1 package com.example.idempotency.idempotency;
2
3 import org.springframework.stereotype.Service;
4 import org.springframework.transaction.annotation.Transactional;
5
6 import java.util.Optional;
7
8 @Service
9 public class IdempotencyService {
10     private final IdempotencyKeyRepository repository;
11
12     public IdempotencyService(IdempotencyKeyRepository
13         repository) {
14         this.repository = repository;
15     }
16
17     public Optional<IdempotencyKeyEntity> findByKey(String key)
18     {
19         return repository.findByIdempotencyKey(key);
20     }
21
22     @Transactional
23     public IdempotencyKeyEntity registerKey(String key, String
24         method, String path) {
25         return repository.save(new IdempotencyKeyEntity(key,
26             method, path));
27     }
28
29     @Transactional
30     public void saveResponse(String key, int status, String
31         responseBody) {
32         repository.findByIdempotencyKey(key).ifPresent(entity
33             -> {
34                 entity.setResponseStatus(status);
35                 entity.setResponseBody(responseBody);
36                 repository.save(entity);
37             });
38     }
39 }
```

A.13. IdempotencyFilter.java

Листинг 13: IdempotencyFilter.java — HTTP-фильтр идиempотентности

```
1 package com.example.idempotency.idempotency;
2
3 import com.example.idempotency.config.IdempotencyProperties;
4 import jakarta.servlet.*;
5 import jakarta.servlet.http.*;
```



```

6 import org.springframework.dao.DataIntegrityViolationException;
7 import org.springframework.http.HttpStatus;
8 import org.springframework.web.filter.OncePerRequestFilter;
9 import org.springframework.web.util.
    ContentCachingResponseWrapper;
10 import java.io.IOException;
11 import java.nio.charset.StandardCharsets;
12
13 public class IdempotencyFilter extends OncePerRequestFilter {
14     public static final String IDEMPOTENCY_KEY_HEADER = "
        Idempotency-Key";
15     private final IdempotencyService idempotencyService;
16     private final IdempotencyProperties properties;
17
18     public IdempotencyFilter(IdempotencyService svc,
19         IdempotencyProperties props) {
20     }
21
22     @Override
23     protected void doFilterInternal(HttpServletRequest request,
24         HttpServletResponse response, FilterChain chain)
25         throws ServletException, IOException {
26         if (!"POST".equalsIgnoreCase(request.getMethod())) {
27             chain.doFilter(request, response); return;
28         }
29         if (!properties.isEnabled()) {
30             chain.doFilter(request, response); return;
31         }
32         String key = request.getHeader(IDEMPOTENCY_KEY_HEADER);
33         if (key == null || key.isBlank()) {
34             response.setStatus(HttpStatus.BAD_REQUEST.value());
35             response.setContentType("application/json;charset=
                UTF-8");
36             response.getWriter().write(
37                 "{\"error\":\"Idempotency-Key required\"}");
38             return;
39         }
40         var existing = idempotencyService.findByKey(key);
41         if (existing.isPresent()) {
42             var cached = existing.get();
43             if (cached.getResponseStatus() != null) {
44                 response.setStatus(cached.getResponseStatus());
45                 response.setContentType("application/json;
                    charset=UTF-8");
46                 if (cached.getResponseBody() != null)
47                     response.getWriter().write(cached.
                        getResponseBody());
48                 return;
49             }
50             response.setStatus(HttpStatus.CONFLICT.value());
51             return;

```

```

52     }
53     try {
54         idempotencyService.registerKey(key,
55             request.getMethod(), request.getRequestURI());
56     } catch (DataIntegrityViolationException e) {
57         response.setStatus(HttpStatus.CONFLICT.value());
58         return;
59     }
60     var wrapper = new ContentCachingResponseWrapper(
61         response);
62     chain.doFilter(request, wrapper);
63     String body = new String(
64         wrapper.getContentAsByteArray(), StandardCharsets.
65             UTF_8);
66     idempotencyService.saveResponse(key, wrapper.getStatus
67         (), body);
68     wrapper.copyBodyToResponse();
69 }

```

A.14. Order.java

Листинг 14: Order.java — сущность заказа

```

1 package com.example.idempotency.order;
2
3 import jakarta.persistence.*;
4 import java.math.BigDecimal;
5 import java.time.LocalDateTime;
6 import java.util.UUID;
7
8 @Entity @Table(name = "orders")
9 public class Order {
10     @Id @GeneratedValue(strategy = GenerationType.UUID)
11     private UUID id;
12     @Column(nullable = false)
13     private String description;
14     @Column(nullable = false, precision = 19, scale = 2)
15     private BigDecimal amount;
16     @Column(nullable = false) @Enumerated(EnumType.STRING)
17     private OrderStatus status = OrderStatus.CREATED;
18     @Column(name = "created_at", nullable = false, updatable =
19         false)
20     private LocalDateTime createdAt = LocalDateTime.now();
21
22     public Order() {}
23     public Order(String description, BigDecimal amount) {
24         this.description = description; this.amount = amount;
25     }
26     // Getters u Setters
27     public UUID getId() { return id; }

```

```

27     public String getDescription() { return description; }
28     public void setDescription(String d) { this.description = d
        ; }
29     public BigDecimal getAmount() { return amount; }
30     public void setAmount(BigDecimal a) { this.amount = a; }
31     public OrderStatus getStatus() { return status; }
32     public void setStatus(OrderStatus s) { this.status = s; }
33 }

```

A.15. OrderStatus.java

Листинг 15: OrderStatus.java

```

1 package com.example.idempotency.order;
2
3 public enum OrderStatus { CREATED, CONFIRMED, CANCELLED }

```

A.16. OrderDto.java

Листинг 16: OrderDto.java — DTO заказа с валидацией

```

1 package com.example.idempotency.order;
2
3 import jakarta.validation.constraints.*;
4 import java.math.BigDecimal;
5
6 public record OrderDto(
7     @NotBlank(message = "Описание обязательно") String
        description,
8     @NotNull @DecimalMin("0.01") BigDecimal amount,
9     OrderStatus status
10 ) {}

```

A.17. OrderRepository.java

Листинг 17: OrderRepository.java

```

1 package com.example.idempotency.order;
2
3 import org.springframework.data.jpa.repository.JpaRepository;
4 import java.util.UUID;
5
6 public interface OrderRepository extends JpaRepository<Order,
    UUID> {}

```

A.18. OrderService.java

ЛИСТИНГ 18: OrderService.java — бизнес-логика заказов

```
1 package com.example.idempotency.order;
2
3 import org.springframework.stereotype.Service;
4 import org.springframework.transaction.annotation.Transactional
5 ;
6 import java.util.*;
7
8 @Service
9 public class OrderService {
10     private final OrderRepository orderRepository;
11
12     public OrderService(OrderRepository repo) { this.
13         orderRepository = repo; }
14
15     public List<Order> findAll() { return orderRepository.
16         findAll(); }
17
18     public Order findById(UUID id) {
19         return orderRepository.findById(id)
20             .orElseThrow(() -> new OrderNotFoundException(id));
21     }
22
23     @Transactional
24     public Order create(OrderDto dto) {
25         return orderRepository.save(new Order(dto.description()
26             , dto.amount()));
27     }
28
29     @Transactional
30     public Order update(UUID id, OrderDto dto) {
31         var order = findById(id);
32         order.setDescription(dto.description());
33         order.setAmount(dto.amount());
34         if (dto.status() != null) order.setStatus(dto.status())
35         ;
36         return orderRepository.save(order);
37     }
38
39     @Transactional
40     public void delete(UUID id) { orderRepository.delete(
41         findById(id)); }
```

```
37 public static class OrderNotFoundException extends
38     RuntimeException {
39     public OrderNotFoundException(UUID id) {
40         super("Order not found: " + id);
41     }
42 }
```

A.19. OrderController.java

Листинг 19: OrderController.java — тип контроллера №1: @RestController

```
1 package com.example.idempotency.order;
2
3 import jakarta.validation.Valid;
4 import org.springframework.http.*;
5 import org.springframework.web.bind.annotation.*;
6 import java.util.*;
7
8 @RestController
9 @RequestMapping("/api/orders")
10 public class OrderController {
11     private final OrderService orderService;
12
13     public OrderController(OrderService svc) { this.
        orderService = svc; }
14
15     @GetMapping
16     public List<Order> list() { return orderService.findAll();
        }
17
18     @GetMapping("/{id}")
19     public Order get(@PathVariable UUID id) { return
        orderService.findById(id); }
20
21     @PostMapping
22     public ResponseEntity<Order> create(@Valid @RequestBody
        OrderDto dto) {
23         return ResponseEntity.status(HttpStatus.CREATED)
24             .body(orderService.create(dto));
25     }
26
27     @PutMapping("/{id}")
28     public Order update(@PathVariable UUID id, @Valid
        @RequestBody OrderDto dto) {
29         return orderService.update(id, dto);
30     }
31
32     @DeleteMapping("/{id}") @ResponseStatus(HttpStatus.
        NO_CONTENT)
33     public void delete(@PathVariable UUID id) { orderService.
        delete(id); }
34 }
```

A.20. Payment.java

Листинг 20: Payment.java — сущность платежа

```
1 package com.example.idempotency.payment;
2
3 import com.example.idempotency.order.Order;
4 import jakarta.persistence.*;
5 import java.math.BigDecimal;
6 import java.time.LocalDateTime;
7 import java.util.UUID;
8
9 @Entity @Table(name = "payments")
10 public class Payment {
11     @Id @GeneratedValue(strategy = GenerationType.UUID)
12     private UUID id;
13     @ManyToOne(fetch = FetchType.EAGER)
14     @JoinColumn(name = "order_id", nullable = false)
15     private Order order;
16     @Column(nullable = false, precision = 19, scale = 2)
17     private BigDecimal amount;
18     @Column(nullable = false) @Enumerated(EnumType.STRING)
19     private PaymentStatus status = PaymentStatus.PENDING;
20     @Column(name = "created_at", nullable = false, updatable =
21         false)
22     private LocalDateTime createdAt = LocalDateTime.now();
23
24     public Payment() {}
25     public Payment(Order order, BigDecimal amount) {
26         this.order = order; this.amount = amount;
27     }
28     // Getters u Setters
29     public UUID getId() { return id; }
30     public BigDecimal getAmount() { return amount; }
31     public PaymentStatus getStatus() { return status; }
```

A.21. PaymentStatus.java

Листинг 21: PaymentStatus.java

```
1 package com.example.idempotency.payment;
2
3 public enum PaymentStatus { PENDING, COMPLETED, FAILED }
```

A.22. PaymentDto.java

Листинг 22: PaymentDto.java

```

1 package com.example.idempotency.payment;
2
3 import java.math.BigDecimal;
4 import java.util.UUID;
5
6 public record PaymentDto(UUID orderId, BigDecimal amount) {}

```

A.23. PaymentRepository.java

Листинг 23: PaymentRepository.java

```

1 package com.example.idempotency.payment;
2
3 import org.springframework.data.jpa.repository.JpaRepository;
4 import java.util.UUID;
5
6 public interface PaymentRepository extends JpaRepository<
    Payment, UUID> {}

```

A.24. PaymentService.java

Листинг 24: PaymentService.java — бизнес-логика платежей

```

1 package com.example.idempotency.payment;
2
3 import com.example.idempotency.order.*;
4 import org.springframework.stereotype.Service;
5 import org.springframework.transaction.annotation.Transactional
6 ;
7 import java.util.*;
8
9 @Service
10 public class PaymentService {
11     private final PaymentRepository paymentRepository;
12     private final OrderRepository orderRepository;
13
14     public PaymentService(PaymentRepository pr, OrderRepository
15         or) {
16         this.paymentRepository = pr; this.orderRepository = or;
17     }
18
19     public List<Payment> findAll() { return paymentRepository.
20         findAll(); }
21
22     public Payment findById(UUID id) {
23         return paymentRepository.findById(id)
24             .orElseThrow(() -> new PaymentNotFoundException(id)
25             );
26     }
27 }

```

```

23
24     @Transactional
25     public Payment create(PaymentDto dto) {
26         var order = orderRepository.findById(dto.orderId())
27             .orElseThrow(() -> new OrderService.
28                 OrderNotFoundException(dto.orderId()));
29         return paymentRepository.save(new Payment(order, dto.
30             amount()));
31     }
32
33     public static class PaymentNotFoundException extends
34         RuntimeException {
35         public PaymentNotFoundException(UUID id) {
36             super("Payment not found: " + id);
37         }
38     }
39 }

```

A.25. PaymentHandler.java

Листинг 25: PaymentHandler.java — тип контроллера №2: HandlerFunction

```

1 package com.example.idempotency.payment;
2
3 import com.fasterxml.jackson.databind.ObjectMapper;
4 import org.springframework.stereotype.Component;
5 import org.springframework.web.servlet.function.*;
6 import java.net.URI;
7 import java.util.UUID;
8
9 @Component
10 public class PaymentHandler {
11     private final PaymentService paymentService;
12     private final ObjectMapper objectMapper;
13
14     public PaymentHandler(PaymentService svc, ObjectMapper om)
15     {
16         this.paymentService = svc; this.objectMapper = om;
17     }
18
19     public ServerResponse list(ServerRequest request) {
20         return ServerResponse.ok().body(paymentService.findAll
21             ());
22     }
23
24     public ServerResponse getById(ServerRequest request) {
25         var id = UUID.fromString(request.pathVariable("id"));
26         try {
27             return ServerResponse.ok().body(paymentService.
28                 findById(id));
29         } catch (PaymentService.PaymentNotFoundException e) {

```



```

27         return ServerResponse.notFound().build();
28     }
29 }
30
31 public ServerResponse create(ServerRequest request) throws
Exception {
32     var dto = objectMapper.readValue(
33         request.servletRequest().getInputStream(),
34         PaymentDto.class);
35     var payment = paymentService.create(dto);
36     return ServerResponse
37         .created(URI.create("/api/payments/" + payment.
38             getId()))
39         .body(payment);

```

A.26. PaymentRouter.java

Листинг 26: PaymentRouter.java — тип контроллера №2: RouterFunction

```

1 package com.example.idempotency.payment;
2
3 import org.springframework.context.annotation.*;
4 import org.springframework.web.servlet.function.*;
5 import static org.springframework.http.MediaType.
6     APPLICATION_JSON;
7 import static org.springframework.web.servlet.function.
8     RequestPredicates.*;
9
10 @Configuration
11 public class PaymentRouter {
12     @Bean
13     public RouterFunction<ServerResponse> paymentRoutes(
14         PaymentHandler h) {
15         return RouterFunctions.route()
16             .path("/api/payments", b -> b
17                 .GET("", accept(APPLICATION_JSON), h::list)
18                 .GET("/{id}", accept(APPLICATION_JSON), h::
19                     getId)
20                 .POST("", contentType(APPLICATION_JSON), h::
21                     create)
22             ).build();
23     }
24 }

```

A.27. Product.java

Листинг 27: Product.java — сущность с @Version для optimistic locking

```
1 package com.example.idempotency.product;
2
3 import jakarta.persistence.*;
4 import java.math.BigDecimal;
5
6 @Entity @Table(name = "products")
7 public class Product {
8     @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
9     private Long id;
10    @Column(nullable = false, unique = true, length = 100)
11    private String sku;
12    @Column(nullable = false)
13    private String name;
14    @Column(nullable = false, precision = 19, scale = 2)
15    private BigDecimal price;
16    @Column(nullable = false)
17    private Integer quantity = 0;
18    @Version
19    private Integer version;
20
21    public Product() {}
22    public Product(String sku, String name, BigDecimal price,
23        Integer qty) {
24        this.sku = sku; this.name = name; this.price = price;
25        this.quantity = qty;
26    }
27    // Getters u Setters
28    public Long getId() { return id; }
29    public String getSku() { return sku; }
30    public String getName() { return name; }
31    public void setName(String n) { this.name = n; }
32    public Integer getQuantity() { return quantity; }
33    public void setQuantity(Integer q) { this.quantity = q; }
34    public Integer getVersion() { return version; }
35 }
```

A.28. ProductRepository.java

Листинг 28: ProductRepository.java — тип контроллера №3: Spring Data REST

```
1 package com.example.idempotency.product;
2
3 import org.springframework.data.jpa.repository.JpaRepository;
4 import org.springframework.data.rest.core.annotation.
5     RepositoryRestResource;
6 import java.util.Optional;
```

```

7 @RepositoryRestResource(collectionResourceRel = "products",
   path = "products")
8 public interface ProductRepository extends JpaRepository<
   Product, Long> {
9     Optional<Product> findBySku(String sku);
10 }

```

A.29. ProductEventHandler.java

Листинг 29: ProductEventHandler.java — обработчик событий Spring Data REST

```

1 package com.example.idempotency.product;
2
3 import org.slf4j.*;
4 import org.springframework.data.rest.core.annotation.*;
5 import org.springframework.stereotype.Component;
6
7 @Component @RepositoryEventHandler
8 public class ProductEventHandler {
9     private static final Logger log =
10         LoggerFactory.getLogger(ProductEventHandler.class);
11
12     @HandleBeforeCreate
13     public void handleBeforeCreate(Product product) {
14         log.info("Creating product: SKU={}", product.getSku());
15         if (product.getQuantity() == null) product.setQuantity
16             (0);
17     }
18
19     @HandleBeforeSave
20     public void handleBeforeSave(Product product) {
21         log.info("Updating product: id={}, version={}",
22             product.getId(), product.getVersion());
23     }
24 }

```

A.30. GlobalExceptionHandler.java

Листинг 30: GlobalExceptionHandler.java — глобальная обработка ошибок

```

1 package com.example.idempotency.exception;
2
3 import com.example.idempotency.order.OrderService;
4 import com.example.idempotency.payment.PaymentService;
5 import org.springframework.dao.DataIntegrityViolationException;
6 import org.springframework.http.*;
7 import org.springframework.orm.
   ObjectOptimisticLockingFailureException;

```

```

8 import org.springframework.web.bind.
   MethodArgumentNotValidException;
9 import org.springframework.web.bind.annotation.*;
10 import java.time.LocalDateTime;
11 import java.util.Map;
12
13 @RestControllerAdvice
14 public class GlobalExceptionHandler {
15
16     @ExceptionHandler(OrderService.OrderNotFoundException.class
17     )
18     public ResponseEntity<Map<String, Object>>
19         handleOrderNotFound(
20             OrderService.OrderNotFoundException e) {
21         return buildError(HttpStatus.NOT_FOUND, e.getMessage())
22             ;
23     }
24
25     @ExceptionHandler(PaymentService.PaymentNotFoundException.
26         class)
27     public ResponseEntity<Map<String, Object>>
28         handlePaymentNotFound(
29             PaymentService.PaymentNotFoundException e) {
30         return buildError(HttpStatus.NOT_FOUND, e.getMessage())
31             ;
32     }
33
34     @ExceptionHandler(DataIntegrityViolationException.class)
35     public ResponseEntity<Map<String, Object>>
36         handleDataIntegrity(
37             DataIntegrityViolationException e) {
38         return buildError(HttpStatus.CONFLICT, "Duplicate
39             resource");
40     }
41
42     @ExceptionHandler(ObjectOptimisticLockingFailureException.
43         class)
44     public ResponseEntity<Map<String, Object>>
45         handleOptimisticLock(
46             ObjectOptimisticLockingFailureException e) {
47         return buildError(HttpStatus.CONFLICT, "Version
48             conflict");
49     }
50
51     @ExceptionHandler(MethodArgumentNotValidException.class)
52     public ResponseEntity<Map<String, Object>> handleValidation
53         (
54             MethodArgumentNotValidException e) {
55         var msg = e.getBindingResult().getFieldErrors().stream
56             ()
57             .findFirst().map(err -> err.getField() + ": "

```

```

45         + err.getDefaultMessage()).orElse("Validation
         error");
46     return buildError(HttpStatus.BAD_REQUEST, msg);
47 }
48
49 private ResponseEntity<Map<String, Object>> buildError(
50     HttpStatus status, String message) {
51     return ResponseEntity.status(status).body(Map.of(
52         "timestamp", LocalDateTime.now().toString(),
53         "status", status.value(),
54         "error", status.getReasonPhrase(),
55         "message", message));
56 }
57 }

```

A.31. BaseIntegrationTest.java

Листинг 31: BaseIntegrationTest.java — базовый класс с Testcontainers

```

1 package com.example.idempotency;
2
3 import org.springframework.boot.test.autoconfigure.web.servlet.
    AutoConfigureMockMvc;
4 import org.springframework.boot.test.context.SpringBootTest;
5 import org.springframework.test.context.*;
6 import org.testcontainers.containers.PostgreSQLContainer;
7
8 @SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.
    RANDOM_PORT)
9 @AutoConfigureMockMvc
10 @ActiveProfiles("test")
11 public abstract class BaseIntegrationTest {
12
13     static final PostgreSQLContainer<?> POSTGRES =
14         new PostgreSQLContainer<>("postgres:16-alpine")
15             .withDatabaseName("idempotency_test")
16             .withUsername("test").withPassword("test");
17
18     static { POSTGRES.start(); }
19
20     @DynamicPropertySource
21     static void configureProperties(DynamicPropertyRegistry
        registry) {
22         registry.add("spring.datasource.url", POSTGRES::
            getJdbcUrl);
23         registry.add("spring.datasource.username", POSTGRES::
            getUsername);
24         registry.add("spring.datasource.password", POSTGRES::
            getPassword);
25     }
26 }

```

A.32. V4__idempotency_content_meta.sql (расширение под произвольный Content-Type)

Листинг 32: V4: расширение idempotency_keys для SOAP/gRPC

```
1 ALTER TABLE idempotency_keys
2     ADD COLUMN response_content_type VARCHAR(150),
3     ADD COLUMN response_kind        VARCHAR(16) NOT NULL
4     DEFAULT 'TEXT',
5     ADD COLUMN response_body_bytes  BYTEA;
```

A.33. ResponseKind.java (дискриминатор тела)

Листинг 33: Enum для различения текстового и бинарного снэпшота

```
1 package com.example.idempotency.idempotency;
2
3 public enum ResponseKind { TEXT, BINARY }
```

A.34. IdempotencyKeyEntity.java (обновлённая после V4)

Листинг 34: JPA-сущность со снэпшотом ответа (extract)

```
1 @Entity
2 @Table(name = "idempotency_keys")
3 public class IdempotencyKeyEntity {
4     @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
5     private Long id;
6
7     @Column(name = "idempotency_key", nullable = false, unique
8             = true)
9     private String idempotencyKey;
10
11     @Column(nullable = false, length = 10) private String
12         method;
13     @Column(nullable = false, length = 500) private String path
14         ;
15
16     @Column(name = "response_status")
17                                     private Integer
18         responseStatus;
19     @Column(name = "response_body", columnDefinition = "TEXT")
20         private String responseBody;
```

```

15     @Column(name = "response_content_type", length = 150)
16         private String responseContentType;
17
18     @Enumerated(EnumType.STRING)
19     @Column(name = "response_kind", nullable = false, length = 16)
20     private ResponseKind responseKind = ResponseKind.TEXT;
21
22     @Basic(fetch = FetchType.LAZY)
23     @Column(name = "response_body_bytes", columnDefinition = "
24         BYTEA")
25     private byte[] responseBodyBytes;
26
27     @Column(name = "created_at", nullable = false, updatable =
28         false)
29     private LocalDateTime createdAt = LocalDateTime.now();
30     @Column(name = "expires_at", nullable = false)
31     private LocalDateTime expiresAt = LocalDateTime.now().
    plusHours(24);
32
33     // getters / setters опущены
34 }

```

A.35. IdempotencyService.java (обновлённый с двумя save-методами)

Листинг 35: Единая точка интеграции для всех транспортов

```

1 @Service
2 public class IdempotencyService {
3     private final IdempotencyKeyRepository repository;
4
5     public IdempotencyService(IdempotencyKeyRepository
6         repository) {
7         this.repository = repository;
8     }
9
10    public Optional<IdempotencyKeyEntity> findByKey(String key)
11    {
12        return repository.findByIdempotencyKey(key);
13    }
14
15    @Transactional
16    public IdempotencyKeyEntity registerKey(String key, String
17        method, String path) {
18        return repository.save(new IdempotencyKeyEntity(key,
19            method, path));
20    }
21
22    /** Сохранение текстового ответа (REST/SOAP/GraphQL). */

```

```

19     @Transactional
20     public void saveTextResponse(String key, int status, String
        contentType, String responseBody) {
21         repository.findByIdempotencyKey(key).ifPresent(entity
            -> {
22             entity.setResponseStatus(status);
23             entity.setResponseContentType(contentType);
24             entity.setResponseKind(ResponseKind.TEXT);
25             entity.setResponseBody(responseBody);
26             repository.save(entity);
27         });
28     }
29
30     /** Сохранение бинарного ответа (gRPC protobuf). */
31     @Transactional
32     public void saveBinaryResponse(String key, int status,
        String contentType, byte[] bytes) {
33         repository.findByIdempotencyKey(key).ifPresent(entity
            -> {
34             entity.setResponseStatus(status);
35             entity.setResponseContentType(contentType);
36             entity.setResponseKind(ResponseKind.BINARY);
37             entity.setResponseBodyBytes(bytes);
38             repository.save(entity);
39         });
40     }
41 }

```

A.36. IdempotencyFilter.java (обновлённый, транспорт-агностичный)

Листинг 36: Фильтр сохраняет реальный Content-Type и пишет байты через OutputStream

```

1 public class IdempotencyFilter extends OncePerRequestFilter {
2     public static final String IDEMPOTENCY_KEY_HEADER = "
        Idempotency-Key";
3     private static final String JSON_UTF8 = "application/json;
        charset=UTF-8";
4
5     private final IdempotencyService idempotencyService;
6     private final IdempotencyProperties properties;
7
8     @Override
9     protected void doFilterInternal(HttpServletRequest request,
10                                     HttpServletResponse
        response,
11                                     FilterChain chain) throws
        ServletException,
        IOException {

```



```

12     if (!"POST".equalsIgnoreCase(request.getMethod()) || !
13         properties.isEnabled()) {
14         chain.doFilter(request, response); return;
15     }
16     String key = request.getHeader(IDEMPOTENCY_KEY_HEADER);
17     if (key == null || key.isBlank()) {
18         writeError(response, HttpStatus.BAD_REQUEST,
19             "{\n\"error\":\n\"Idempotency-Key header is
20             required for POST requests\n}\"");
21         return;
22     }
23     var existing = idempotencyService.findByKey(key);
24     if (existing.isPresent()) {
25         var cached = existing.get();
26         if (cached.getResponseStatus() != null) { replay(
27             response, cached); return; }
28         writeError(response, HttpStatus.CONFLICT,
29             "{\n\"error\":\n\"Request with this key is
30             already being processed\n}\"");
31         return;
32     }
33     try { idempotencyService.registerKey(key, request.
34         getMethod(), request.getRequestURI()); }
35     catch (DataIntegrityViolationException e) {
36         writeError(response, HttpStatus.CONFLICT,
37             "{\n\"error\":\n\"Request with this key is
38             already being processed\n}\"");
39         return;
40     }
41     ContentCachingResponseWrapper wrapper = new
42         ContentCachingResponseWrapper(response);
43     chain.doFilter(request, wrapper);
44     String contentType = resolveContentType(wrapper);
45     String body = new String(wrapper.getContentAsByteArray
46         (), StandardCharsets.UTF_8);
47     idempotencyService.saveTextResponse(key, wrapper.
48         getStatus(), contentType, body);
49     wrapper.copyBodyToResponse();
50 }
51
52 private static void replay(HttpServletResponse response,
53     IdempotencyKeyEntity cached) throws IOException {
54     response.setStatus(cached.getResponseStatus());
55     String ct = cached.getResponseContentType() != null ?
56         cached.getResponseContentType() : JSON_UTF8;
57     response.setContentType(ct);
58     byte[] bytes = (cached.getResponseKind() ==
59         ResponseKind.BINARY && cached.getResponseBodyBytes()
60             != null)
61         ? cached.getResponseBodyBytes()
62         : (cached.getResponseBody() != null ? cached.
63             getResponseBody().getBytes(StandardCharsets.

```

```

        UTF_8) : new byte[0]);
50     response.setContentLength(bytes.length);
51     response.getOutputStream().write(bytes);
52     response.getOutputStream().flush();
53 }
54
55 private static String resolveContentType(
    ContentCachingResponseWrapper w) {
56     String h = w.getHeader(HttpHeaders.CONTENT_TYPE);
57     if (h != null && !h.isBlank()) return h;
58     String ct = w.getContentType();
59     return (ct != null && !ct.isBlank()) ? ct : JSON_UTF8;
60 }
61 }

```

A.37. WebMvcConfig.java (URL-паттерны фильтра для всех четырёх HTTP-транспортов)

Листинг 37: Единый бин IdempotencyFilter перехватывает 4 транспорта

```

1 @Configuration
2 @EnableConfigurationProperties(IdempotencyProperties.class)
3 public class WebMvcConfig {
4     @Bean
5     public FilterRegistrationBean<IdempotencyFilter>
        idempotencyFilterRegistration(
6         IdempotencyService svc, IdempotencyProperties props
7     ) {
8         var reg = new FilterRegistrationBean<IdempotencyFilter>
9             >();
10        reg.setFilter(new IdempotencyFilter(svc, props));
11        reg.addUrlPatterns(
12            "/api/orders/*", "/api/orders",
13            "/api/payments/*", "/api/payments",
14            "/api/soap/*",
15            "/graphql");
16        reg.setOrder(1);
17        return reg;
18    }
19 }

```

A.38. V5__shipments.sql + xsd/shipments.xsd (SOAP-домен)

Листинг 38: V5 миграция и XSD контракт SOAP

```

1 -- V5__shipments.sql
2 CREATE TABLE shipments (

```

```

3      id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
4      tracking_number    VARCHAR(64) NOT NULL,
5      recipient          VARCHAR(255) NOT NULL,
6      status             VARCHAR(32) NOT NULL DEFAULT 'CREATED',
7      created_at         TIMESTAMP NOT NULL DEFAULT NOW(),
8      CONSTRAINT uk_shipment_tracking_number UNIQUE (
9          tracking_number)
);

```

Листинг 39: xsd/shipments.xsd

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"
3           xmlns:tns="http://example.com/idempotency/shipments"
4           targetNamespace="http://example.com/idempotency/
5               shipments"
6           elementFormDefault="qualified">
7     <xs:element name="createShipmentRequest">
8       <xs:complexType><xs:sequence>
9         <xs:element name="trackingNumber" type="xs:string"/
10        >
11         <xs:element name="recipient" type="xs:string"/>
12       </xs:sequence></xs:complexType>
13     </xs:element>
14     <xs:element name="createShipmentResponse">
15       <xs:complexType><xs:sequence>
16         <xs:element name="id" type="xs:string"/>
17         <xs:element name="trackingNumber" type="xs:string"/
18         >
19         <xs:element name="status" type="xs:string"/>
20       </xs:sequence></xs:complexType>
21     </xs:element>
22 </xs:schema>

```

A.39. ShipmentEndpoint.java + WebServiceConfig.java

Листинг 40: SOAP-эндпоинт и регистрация MessageDispatcherServlet

```

1 @Endpoint
2 public class ShipmentEndpoint {
3     public static final String NAMESPACE = "http://example.com/
4         idempotency/shipments";
5     private final ShipmentService service;
6
7     @PayloadRoot(namespace = NAMESPACE, localPart = "
8         createShipmentRequest")
9     @ResponsePayload
10    public DOMSource createShipment(@RequestPayload DOMSource
11        request) throws Exception {
12        Element root = rootElement(request);
13    }
14 }

```

```

10     Shipment created = service.create(childText(root, "
11         trackingNumber"),
12                                     childText(root, "
13         recipient"));
14
15     Document doc = newDocument();
16     Element resp = doc.createElementNS(NAMESPACE, "
17         createShipmentResponse");
18     doc.appendChild(resp);
19     appendChild(doc, resp, "id", created.getId().toString()
20     );
21     appendChild(doc, resp, "trackingNumber", created.
22         getTrackingNumber());
23     appendChild(doc, resp, "status", created.getStatus());
24     return new DOMSource(doc);
25 }
26 // rootElement / childText / appendChild / newDocument onyw
27     eHbI
28 }
29
30 @EnableWs @Configuration
31 public class WebServiceConfig extends WsConfigurerAdapter {
32     @Bean
33     public ServletRegistrationBean<MessageDispatcherServlet>
34         messageDispatcherServlet(
35         ApplicationContext ctx) {
36         MessageDispatcherServlet servlet = new
37             MessageDispatcherServlet();
38         servlet.setApplicationContext(ctx);
39         servlet.setTransformWsdlLocations(true);
40         return new ServletRegistrationBean<>(servlet, "/api/
41             soap/*");
42     }
43     @Bean(name = "shipments")
44     public DefaultWsd11Definition shipmentsWsd1(XsdSchema
45         shipmentsSchema) {
46         DefaultWsd11Definition wsdl = new
47             DefaultWsd11Definition();
48         wsdl.setPortTypeName("ShipmentsPort");
49         wsdl.setLocationUri("/api/soap");
50         wsdl.setTargetNamespace(ShipmentEndpoint.NAMESPACE);
51         wsdl.setSchema(shipmentsSchema);
52         return wsdl;
53     }
54     @Bean
55     public XsdSchema shipmentsSchema() {
56         return new SimpleXsdSchema(new ClassPathResource("xsd/
57             shipments.xsd"));
58     }
59 }

```

A.40. notification.proto + V6__notifications.sql

Листинг 41: Контракт gRPC и таблица домена

```
1 // src/main/proto/notification.proto
2 syntax = "proto3";
3 package com.example.idempotency.grpc.notification;
4 option java_multiple_files = true;
5 option java_package = "com.example.idempotency.grpc.
   notification.proto";
6
7 service NotificationService {
8     rpc Send(SendRequest) returns (SendResponse);
9     rpc List(ListRequest) returns (ListResponse);
10 }
11 message SendRequest { string recipient = 1; string message =
   2; }
12 message SendResponse { string id = 1; string status = 2; }
13 message ListRequest {}
14 message ListResponse { repeated NotificationItem items = 1; }
15 message NotificationItem {
16     string id = 1; string recipient = 2; string message = 3;
17     string status = 4;
18 }
```

Листинг 42: V6__notifications.sql

```
1 CREATE TABLE notifications (
2     id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
3     recipient         VARCHAR(255) NOT NULL,
4     message           TEXT NOT NULL,
5     message_hash      VARCHAR(64) NOT NULL,
6     status            VARCHAR(32) NOT NULL DEFAULT 'SENT',
7     created_at        TIMESTAMP NOT NULL DEFAULT NOW(),
8     CONSTRAINT uk_notification_recipient_hash UNIQUE (recipient
9     , message_hash)
10 );
```

A.41. GrpcIdempotencyInterceptor.java (главная часть работы)

Листинг 43: Transport-generic перехватчик: переиспользует IdempotencyService

```
1 @GrpcGlobalServerInterceptor
2 public class GrpcIdempotencyInterceptor implements
   ServerInterceptor {
3     private static final Metadata.Key<String> IDEMPOTENCY_KEY =
4         Metadata.Key.of("idempotency-key", Metadata.
5             ASCII_STRING_MARSHALLER);
```

```

5 private static final String CONTENT_TYPE = "application/
6     grpc+proto";
7
8 private final IdempotencyService idempotencyService;
9 private final IdempotencyProperties properties;
10
11 @Override
12 public <ReqT, RespT> ServerCall.Listener<ReqT>
13     interceptCall(
14         ServerCall<ReqT, RespT> call, Metadata headers,
15         ServerCallHandler<ReqT, RespT> next) {
16
17     if (!properties.isEnabled()) return next.startCall(call
18         , headers);
19
20     String key = headers.get(IDEMPOTENCY_KEY);
21     if (key == null || key.isBlank()) {
22         call.close(Status.INVALID_ARGUMENT.withDescription(
23             "idempotency-key metadata is required"),
24             new Metadata());
25         return noopListener();
26     }
27     var existing = idempotencyService.findByKey(key);
28     if (existing.isPresent()) {
29         IdempotencyKeyEntity cached = existing.get();
30         if (cached.getResponseStatus() != null && cached.
31             getResponseBodyBytes() != null) {
32             try {
33                 RespT msg = call.getMethodDescriptor().
34                     getResponseMarshaller()
35                         .parse(new ByteArrayInputStream(
36                             cached.getResponseBodyBytes()));
37                 call.sendHeaders(new Metadata());
38                 call.sendMessage(msg);
39                 call.close(Status.OK, new Metadata());
40             } catch (Exception e) {
41                 call.close(Status.INTERNAL.withDescription(
42                     "replay failed: " + e.getMessage()),
43                     new Metadata());
44             }
45             return noopListener();
46         }
47         call.close(Status.ABORTED, new Metadata());
48         return noopListener();
49     }
50     try { idempotencyService.registerKey(key, "GRPC", call.
51         getMethodDescriptor().getFullMethodName()); }
52     catch (DataIntegrityViolationException e) {
53         call.close(Status.ALREADY_EXISTS, new Metadata());
54         return noopListener();
55     }
56 }

```

```

47         return next.startCall(new CachingServerCall<>(call, key
48             ), headers);
49     }
50     private static <ReqT> ServerCall.Listener<ReqT>
51         noopListener() {
52         return new ServerCall.Listener<>() {};
53     }
54     private final class CachingServerCall<ReqT, RespT>
55         extends ForwardingServerCall.
56             SimpleForwardingServerCall<ReqT, RespT> {
57     private final String key;
58     CachingServerCall(ServerCall<ReqT, RespT> delegate,
59         String key) {
60         super(delegate); this.key = key;
61     }
62     @Override
63     public void sendMessage(RespT message) {
64         try {
65             byte[] bytes = getMethodDescriptor().
66                 getResponseMarshaller()
67                     .stream(message).readAllBytes();
68             idempotencyService.saveBinaryResponse(key,
69                 Status.OK.getStatusCode().value(),
70                 CONTENT_TYPE, bytes);
71         } catch (IOException e) { /* log only */ }
72         super.sendMessage(message);
73     }
74 }

```

A.42. NotificationGrpcService.java

Листинг 44: Реализация gRPC-сервиса (@GrpcService)

```

1 @GrpcService
2 public class NotificationGrpcService
3     extends NotificationServiceGrpc.
4         NotificationServiceImplBase {
5     private final NotificationService service;
6     private final NotificationRepository repository;
7
8     @Override
9     public void send(SendRequest req, StreamObserver<
10         SendResponse> observer) {
11         Notification n = service.send(req.getRecipient(), req.
12             getMessage());
13         observer.onNext(SendResponse.newBuilder()
14             .setId(n.getId().toString()).setStatus(n.
15                 getStatus()).build());
16     }
17 }

```

```

12         observer.onCompleted();
13     }
14
15     @Override
16     public void list(ListRequest req, StreamObserver<
17         ListResponse> observer) {
18         ListResponse.Builder b = ListResponse.newBuilder();
19         repository.findAll().forEach(n -> b.addItem(
20             NotificationItem.newBuilder()
21                 .setId(n.getId().toString()).setRecipient(n.
22                     getRecipient())
23                 .setMessage(n.getMessage()).setStatus(n.
24                     getStatus()).build()));
25         observer.onNext(b.build());
26         observer.onCompleted();
27     }
28 }

```

A.43. schema.graphqls + V7__inventories.sql

Листинг 45: GraphQL-схема (SDL) и таблица домена

```

1 # src/main/resources/graphql/schema.graphqls
2 type Inventory { id: ID!, itemName: String!, quantity: Int! }
3 type Query { inventories: [Inventory!]!, inventory(id: ID!):
4     Inventory }
5 type Mutation { reserveInventory(input: ReserveInput!):
6     Inventory! }
7 input ReserveInput { itemName: String!, quantity: Int! }

```

Листинг 46: V7__inventories.sql

```

1 CREATE TABLE inventories (
2     id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
3     item_name   VARCHAR(255) NOT NULL,
4     quantity    INTEGER NOT NULL,
5     created_at  TIMESTAMP NOT NULL DEFAULT NOW(),
6     CONSTRAINT uk_inventory_item_name UNIQUE (item_name)
7 );

```

A.44. InventoryGraphqlController.java

Листинг 47: GraphQL-контроллер с @QueryMapping / @MutationMapping

```

1 @Controller
2 public class InventoryGraphqlController {
3     private final InventoryRepository repository;
4
5     @QueryMapping

```



```

6      public List<Inventory> inventories() { return repository.
          findAll(); }
7
8      @QueryMapping
9      public Inventory inventory(@Argument String id) {
10         return repository.findById(UUID.fromString(id)).orElse(
            null);
11     }
12
13     @MutationMapping
14     @Transactional
15     public Inventory reserveInventory(@Argument ReserveInput
        input) {
16         return repository.save(new Inventory(input.itemName(),
            input.quantity()));
17     }
18
19     public record ReserveInput(String itemName, int quantity)
        {}
20 }

```

A.45. V8__drop_new_constraints.sql (профиль non-idempotent)

Листинг 48: Дроп UNIQUE на трёх новых доменах — демонстрация багов

```

1 ALTER TABLE shipments      DROP CONSTRAINT IF EXISTS
    uk_shipment_tracking_number;
2 ALTER TABLE notifications DROP CONSTRAINT IF EXISTS
    uk_notification_recipient_hash;
3 ALTER TABLE inventories   DROP CONSTRAINT IF EXISTS
    uk_inventory_item_name;

```